

# VDerm-X



Session: 2021 - 2025

## **Submitted by:**

Muhammad Moaz    2021-CS-101  
Maryam Siddique    2021-CS-180  
Fiza Azhar    2021-CS-193  
Alishba Javed    2021-CS-205

## **Supervised by:**

Prof. Dr. Muhammad Shoaib

## **Co-Supervised by:**

Ms. Abqa Javed

Department of Computer Science

**University of Engineering and Technology**

**Lahore Pakistan**

# VDerm-X

Submitted to the faculty of the Computer Science Department of the University  
of Engineering and Technology Lahore in partial fulfillment of the requirements  
for the Degree of

## Bachelor of Science in Computer Science.

### Internal Examiner

Signature:

---

Name:

---

Designation:

---

### Chairman

Signature:

---

Prof. Dr. M. Usman Ghani Khan

### External Examiner

Signature:

---

Name:

---

Designation:

---

### Dean

Signature:

---

Prof. Dr. M. Shoaib

Department of Computer Science

**University of Engineering and Technology**  
**Lahore Pakistan**

# Declaration

We declare that the work contained in this thesis is ours, except where explicitly stated otherwise. In addition this work has not been submitted to obtain another degree or professional qualification.

**Name:** Alishba Javeed

**Date:** \_\_\_\_\_

**Signature:** \_\_\_\_\_

**Name:** Fiza Azhar

**Date:** \_\_\_\_\_

**Signature:** \_\_\_\_\_

**Name:** Maryam Siddique

**Date:** \_\_\_\_\_

**Signature:** \_\_\_\_\_

**Name:** Muhammad Moaz

**Date:** \_\_\_\_\_

**Signature:** \_\_\_\_\_

# Acknowledgments

All praise to Allah Almighty, whose guidance and blessings enabled us to successfully complete this project. We extend our deepest gratitude to our supervisor, Prof. Dr. M. Shoaib, and our co-supervisor, Ma'am Abqa, for their invaluable support, encouragement, and expert guidance throughout the development of VDerm-X. Their insights and mentorship played a pivotal role in refining our ideas and transforming them into a practical, impactful solution.

We are also sincerely thankful to our families for their unconditional love, support, and prayers. A special thanks to our fellow team members for their dedication, collaboration, and relentless effort. This project is a reflection of our shared commitment to learning, innovation, and creating meaningful change.

# Executive Summary

This thesis presents the design, development, and evaluation of **VDerm-X**, an AI-driven mobile application aimed at the early detection and management of Lumpy Skin Disease (LSD) in cattle. Recognizing the significant economic and agricultural impact of LSD, particularly in rural regions, this project leverages cutting-edge machine learning, natural language processing, and mobile technologies to deliver a comprehensive, accessible solution for farmers, veterinary professionals, and public health authorities.

The project follows an Agile development methodology, enabling iterative improvements and continuous stakeholder feedback. The disease detection component employs a MobileNetV2-based deep learning model, achieving over 92% accuracy in identifying LSD from cattle images. To complement this, a domain-specific AI chatbot offers real-time treatment guidance and preventive advice, enhancing user support. The application further integrates geolocation features, providing real-time outbreak visualization through interactive heatmaps, thereby equipping both farmers and authorities with vital insights for disease management.

Key innovations include offline functionality for uninterrupted use in low-connectivity areas, a secure veterinary appointment booking system, and robust authentication mechanisms ensuring data privacy and user trust. Extensive testing and evaluation demonstrate the system's effectiveness in real-world settings, showcasing its potential as a transformative tool for improving livestock health outcomes.

In conclusion, **VDerm-X** represents a successful convergence of technology and veterinary science, offering a scalable, adaptable platform to address pressing challenges in animal health management. Future enhancements, such as expanded disease coverage and advanced analytics, promise to further amplify the app's impact and extend its utility across diverse agricultural contexts.

*To all farmers, veterinarians, and innovators striving to  
improve animal health through technology.*

# Contents

|                                                                                                     |          |
|-----------------------------------------------------------------------------------------------------|----------|
| Acknowledgments                                                                                     | iii      |
| List of Figures                                                                                     | xi       |
| List of Tables                                                                                      | xiii     |
| Abbreviations                                                                                       | xiv      |
| Abstract                                                                                            | xv       |
| <b>1 Introduction</b>                                                                               | <b>1</b> |
| 1.1 Project Background . . . . .                                                                    | 1        |
| 1.1.1 Introduction . . . . .                                                                        | 1        |
| 1.1.2 The Challenge of Lumpy Skin Disease . . . . .                                                 | 1        |
| 1.1.3 Limitations of Traditional Detection Methods . . . . .                                        | 1        |
| 1.1.4 The Role of Technology in Disease Detection . . . . .                                         | 2        |
| 1.1.5 The VDerm-X Initiative . . . . .                                                              | 2        |
| 1.1.6 Objectives of the Project . . . . .                                                           | 2        |
| 1.1.7 Significance of the Project . . . . .                                                         | 2        |
| 1.1.8 Why We Choose . . . . .                                                                       | 3        |
| 1.2 Problem Statement . . . . .                                                                     | 3        |
| 1.2.1 Contextual Background . . . . .                                                               | 3        |
| 1.2.2 Identification of the Problem . . . . .                                                       | 3        |
| 1.2.3 Significance of the Problem . . . . .                                                         | 3        |
| 1.2.4 Consequences of the Problem . . . . .                                                         | 4        |
| 1.2.5 Existing Approaches and Limitations . . . . .                                                 | 4        |
| 1.2.6 Research Gap . . . . .                                                                        | 4        |
| 1.2.7 Problem Statement Summary . . . . .                                                           | 4        |
| 1.3 Objectives . . . . .                                                                            | 5        |
| 1.3.1 Development of an AI-Powered Mobile Application for LSD<br>Detection . . . . .                | 5        |
| 1.3.2 Integration of an AI-Driven Chatbot for Treatment Recom-<br>mendations . . . . .              | 5        |
| 1.3.3 Implementation of a Secure Authentication System with Role-<br>Based Access Control . . . . . | 5        |

|          |                                                                              |           |
|----------|------------------------------------------------------------------------------|-----------|
| 1.3.4    | Incorporation of Geolocation Features for Disease Outbreak Mapping . . . . . | 6         |
| 1.3.5    | Enhancement of User Experience through Intuitive Interface Design . . . . .  | 6         |
| 1.3.6    | Establishment of a Feedback Mechanism for Continuous Improvement . . . . .   | 6         |
| 1.3.7    | Promotion of Collaborative Networks among Stakeholders . . . . .             | 6         |
| 1.4      | Scope and Limitations . . . . .                                              | 7         |
| 1.4.1    | Scope of the Project . . . . .                                               | 7         |
| 1.4.2    | Limitations of the Project . . . . .                                         | 7         |
| 1.5      | Methodology Overview . . . . .                                               | 8         |
| 1.5.1    | Research Design . . . . .                                                    | 8         |
| 1.5.2    | Requirement Analysis . . . . .                                               | 9         |
| 1.5.3    | Core Functionalities . . . . .                                               | 9         |
| 1.5.4    | Technology Stack . . . . .                                                   | 9         |
| 1.5.5    | Machine Learning Model Evaluation . . . . .                                  | 9         |
| 1.5.6    | Model Training and Deployment . . . . .                                      | 9         |
| 1.5.7    | AI Chatbot Integration . . . . .                                             | 10        |
| 1.5.8    | Authentication and Role Management . . . . .                                 | 10        |
| 1.5.9    | Frontend Development . . . . .                                               | 10        |
| 1.5.10   | Backend Development . . . . .                                                | 10        |
| 1.5.11   | Testing and Optimization . . . . .                                           | 10        |
| 1.5.12   | Future Enhancements . . . . .                                                | 10        |
| 1.6      | Report Organization . . . . .                                                | 12        |
| <b>2</b> | <b>Software Requirements Specification (SRS)</b>                             | <b>13</b> |
| 2.1      | Software Requirements Specification (SRS) . . . . .                          | 13        |
| 2.1.1    | Purpose . . . . .                                                            | 13        |
| 2.1.2    | Intended Audience . . . . .                                                  | 13        |
| 2.1.3    | Definitions and Acronyms . . . . .                                           | 14        |
| 2.2      | Overall Description . . . . .                                                | 15        |
| 2.2.1    | Product Perspective . . . . .                                                | 15        |
| 2.2.2    | User Characteristics . . . . .                                               | 16        |
| 2.2.3    | Assumptions and Dependencies . . . . .                                       | 17        |
| 2.3      | Specific Requirements . . . . .                                              | 17        |
| 2.3.1    | Functional Requirements . . . . .                                            | 17        |
| 2.3.1.1  | User Registration and Authentication . . . . .                               | 17        |
| 2.3.1.2  | Image Capture and Analysis . . . . .                                         | 17        |
| 2.3.1.3  | Chatbot for Treatment Recommendations . . . . .                              | 18        |
| 2.3.1.4  | Geolocation and Disease Mapping . . . . .                                    | 18        |
| 2.3.1.5  | Notifications and Alerts . . . . .                                           | 18        |
| 2.3.1.6  | Data Management . . . . .                                                    | 18        |
| 2.3.2    | Non-Functional Requirements . . . . .                                        | 18        |
| 2.3.2.1  | Usability . . . . .                                                          | 18        |

---

|          |                                       |           |
|----------|---------------------------------------|-----------|
| 2.3.2.2  | Performance                           | 19        |
| 2.3.2.3  | Security                              | 19        |
| 2.3.2.4  | Reliability                           | 19        |
| 2.3.2.5  | Maintainability                       | 19        |
| 2.3.2.6  | Portability                           | 19        |
| 2.3.3    | Interface Requirements                | 19        |
| 2.3.3.1  | User Interfaces                       | 20        |
| 2.3.3.2  | Hardware Interfaces                   | 20        |
| 2.3.3.3  | Software Interfaces                   | 20        |
| 2.3.3.4  | Communications Interfaces             | 20        |
| <b>3</b> | <b>System Design</b>                  | <b>21</b> |
| 3.1      | Architecture Diagram                  | 21        |
| 3.2      | Use Case Diagram                      | 22        |
| 3.3      | Class Diagram                         | 26        |
| 3.4      | Sequence Diagram                      | 27        |
| 3.5      | UI Mockups                            | 28        |
| 3.6      | Database Diagram                      | 30        |
| <b>4</b> | <b>Implementation</b>                 | <b>31</b> |
| 4.1      | Development Tools & Technologies      | 31        |
| 4.1.1    | Frontend Development                  | 31        |
| 4.1.2    | Backend Development                   | 32        |
| 4.1.3    | Database Management                   | 32        |
| 4.1.4    | Machine Learning Integration          | 32        |
| 4.1.5    | Chatbot System                        | 33        |
| 4.1.6    | Geolocation and Mapping               | 33        |
| 4.1.7    | Veterinary Appointment System         | 33        |
| 4.1.8    | Version Control and CI/CD             | 33        |
| 4.1.9    | Development Environment               | 34        |
| 4.2      | Description of Modules                | 34        |
| 4.2.1    | User Authentication Module            | 34        |
| 4.2.2    | Disease Detection Module              | 34        |
| 4.2.3    | Chatbot Assistance Module             | 35        |
| 4.2.4    | Outbreak Mapping and Reporting Module | 35        |
| 4.2.5    | Veterinary Appointment Booking Module | 35        |
| 4.3      | Key Implementation Details            | 36        |
| 4.3.1    | Machine Learning Optimization         | 36        |
| 4.3.2    | Security and Authentication           | 36        |
| 4.3.3    | On-Device Image Processing            | 36        |
| 4.3.4    | Geolocation and Visualization         | 36        |
| 4.3.5    | Appointment Booking and Notifications | 37        |
| 4.4      | Sample Screenshots                    | 37        |
| <b>5</b> | <b>Testing</b>                        | <b>40</b> |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 5.1      | Testing Strategy . . . . .                               | 40        |
| 5.2      | Test Cases and Results . . . . .                         | 41        |
| 5.3      | Bug Fixes / Issue Tracker . . . . .                      | 43        |
| 5.4      | Tools Used . . . . .                                     | 43        |
| <b>6</b> | <b>Deployment</b>                                        | <b>48</b> |
| 6.1      | Deployment Steps . . . . .                               | 48        |
| 6.1.1    | Hosting Platforms . . . . .                              | 48        |
| 6.1.2    | Dependencies . . . . .                                   | 50        |
| 6.2      | User Instructions . . . . .                              | 51        |
| 6.2.1    | Login and Account Creation . . . . .                     | 51        |
| 6.2.2    | Navigating the App . . . . .                             | 51        |
| 6.2.3    | Using the Disease Detection Feature . . . . .            | 52        |
| 6.2.4    | Booking Veterinary Appointments . . . . .                | 52        |
| 6.3      | Maintenance Guidelines . . . . .                         | 53        |
| 6.3.1    | Frontend Maintenance . . . . .                           | 53        |
| 6.3.2    | Backend Maintenance . . . . .                            | 53        |
| 6.3.3    | Machine Learning Model Maintenance . . . . .             | 53        |
| 6.3.4    | Security and Data Protection . . . . .                   | 54        |
| <b>7</b> | <b>Results, Evaluation &amp; Conclusion</b>              | <b>55</b> |
| 7.1      | Summary of Achievements . . . . .                        | 55        |
| 7.1.1    | AI-Based Disease Detection . . . . .                     | 55        |
| 7.1.2    | Interactive Veterinary Chatbot . . . . .                 | 56        |
| 7.1.3    | Seamless Appointment Booking System . . . . .            | 57        |
| 7.1.4    | Outbreak Mapping and Surveillance . . . . .              | 58        |
| 7.1.5    | Robust Offline Functionality . . . . .                   | 59        |
| 7.2      | Challenges Faced . . . . .                               | 59        |
| 7.2.1    | Data Availability and Quality . . . . .                  | 60        |
| 7.2.2    | Model Optimization for Mobile Devices . . . . .          | 60        |
| 7.2.3    | Integration of Chatbot . . . . .                         | 61        |
| 7.2.4    | User Interface Design . . . . .                          | 62        |
| 7.3      | Lessons Learned . . . . .                                | 63        |
| 7.3.1    | Importance of Data Quality and Diversity . . . . .       | 63        |
| 7.3.2    | User-Centered Design . . . . .                           | 64        |
| 7.3.3    | Offline Functionality is Crucial . . . . .               | 64        |
| 7.3.4    | Scalability Considerations . . . . .                     | 65        |
| 7.3.5    | Cross-Disciplinary Collaboration . . . . .               | 65        |
| 7.4      | Future Enhancements . . . . .                            | 66        |
| 7.4.1    | Improved Machine Learning Models . . . . .               | 66        |
| 7.4.2    | Expanded Disease Detection Capabilities . . . . .        | 67        |
| 7.4.3    | Enhanced User Support Features . . . . .                 | 67        |
| 7.4.4    | Integration of Remote Veterinary Services . . . . .      | 68        |
| 7.4.5    | Further Optimization for Offline Functionality . . . . . | 68        |

---

|                                                      |           |
|------------------------------------------------------|-----------|
| 7.4.6 Push Notifications for Timely Alerts . . . . . | 69        |
| 7.5 Conclusion . . . . .                             | 70        |
| <br>                                                 |           |
| <b>References</b>                                    | <b>74</b> |
| References . . . . .                                 | 74        |
| <br>                                                 |           |
| <b>Final Year Project Proposal Copy</b>              | <b>75</b> |
| <br>                                                 |           |
| <b>Plagiarism Report</b>                             | <b>77</b> |
| <br>                                                 |           |
| <b>Code Snippets</b>                                 | <b>79</b> |
| <br>                                                 |           |
| <b>GitHub Repository Link</b>                        | <b>86</b> |

# List of Figures

|      |                                                                                                                |    |
|------|----------------------------------------------------------------------------------------------------------------|----|
| 1.1  | Methodology Diagram . . . . .                                                                                  | 11 |
| 3.1  | System Architecture Diagram of VDERM-X. . . . .                                                                | 21 |
| 3.2  | A Generic Use Case Diagram for VDERM-X Showing Authentication. . . . .                                         | 22 |
| 3.3  | A Use Case Diagram for VDERM-X, Showing The Extended Interaction Between the System and Cattle Owners. . . . . | 23 |
| 3.4  | A Use Case Diagram for VDERM-X, Showing the Extended Interaction Between the System and its Actors. . . . .    | 24 |
| 3.5  | A Use Case Diagram for VDERM-X, Showing the Extended Interaction Between the System and its Actors. . . . .    | 25 |
| 3.6  | Class Diagram for VDERM-X, Modelling its Object-Oriented Structure. . . . .                                    | 26 |
| 3.7  | Sequence Diagram VDERM-X . . . . .                                                                             | 27 |
| 3.8  | First Screen . . . . .                                                                                         | 28 |
| 3.9  | Chatbot Screen . . . . .                                                                                       | 28 |
| 3.10 | User Side Vet Appointment Screen . . . . .                                                                     | 28 |
| 3.11 | OTP Verification Screen . . . . .                                                                              | 28 |
| 3.12 | Register Screen . . . . .                                                                                      | 29 |
| 3.13 | Vet Appointment Screen . . . . .                                                                               | 29 |
| 3.14 | Database Diagram of VDERM-X. . . . .                                                                           | 30 |
| 4.1  | Launch Screen of the VDERM-X . . . . .                                                                         | 37 |
| 4.2  | Registration Screen of the VDERM-X . . . . .                                                                   | 37 |
| 4.3  | OTP Verification Screen of the VDERM-X . . . . .                                                               | 38 |
| 4.4  | Chatbot Screen of the VDERM-X . . . . .                                                                        | 38 |
| 4.5  | Image Detection Screen of the VDERM-X . . . . .                                                                | 38 |
| 4.6  | Disease Outbreak Screen of the VDERM-X . . . . .                                                               | 38 |
| 4.7  | Vet Appointment History . . . . .                                                                              | 39 |
| 4.8  | Vet Scheduled Appointments . . . . .                                                                           | 39 |
| 4.9  | Vet Slots Addition . . . . .                                                                                   | 39 |
| 5.1  | Signup API Tested Using Postman . . . . .                                                                      | 44 |
| 5.2  | Login API Tested Using Postman . . . . .                                                                       | 44 |
| 5.3  | Disease Detection API Tested Using Postman . . . . .                                                           | 44 |
| 5.4  | Verify OTP API Tested Using Postman . . . . .                                                                  | 45 |
| 5.5  | Resend Otp . . . . .                                                                                           | 45 |

---

|     |                                           |    |
|-----|-------------------------------------------|----|
| 5.6 | Vet Approval API . . . . .                | 45 |
| 5.7 | Chatbott API . . . . .                    | 46 |
| 5.8 | Get all vets API . . . . .                | 46 |
| 5.9 | Get Appointment of specific vet . . . . . | 47 |
| 1   | Plagiarism Report for VDerm-X . . . . .   | 77 |
| 2   | Plagiarism Report for VDerm-X . . . . .   | 78 |

# List of Tables

|                                        |    |
|----------------------------------------|----|
| 5.1 Comprehensive Test Cases . . . . . | 42 |
|----------------------------------------|----|

# Abbreviations

|              |                                                                              |
|--------------|------------------------------------------------------------------------------|
| <b>RBAC</b>  | <b>R</b> ole <b>B</b> ased <b>A</b> ccess <b>C</b> ontrol                    |
| <b>LSD</b>   | <b>L</b> umpy <b>S</b> kin <b>D</b> isease                                   |
| <b>OTP</b>   | <b>O</b> ne <b>T</b> ime <b>P</b> assword                                    |
| <b>CNN</b>   | <b>C</b> onvolutional <b>N</b> eural <b>N</b> etwork                         |
| <b>HTTPS</b> | <b>H</b> yper <b>T</b> ext <b>T</b> ransfer <b>P</b> rotocol <b>S</b> ecure  |
| <b>SRS</b>   | <b>S</b> oftware <b>R</b> equirements <b>S</b> pecification                  |
| <b>AI</b>    | <b>A</b> rtificial <b>I</b> ntelligence                                      |
| <b>API</b>   | <b>A</b> pplication <b>P</b> rogramming <b>I</b> nterface                    |
| <b>GUI</b>   | <b>G</b> raphical <b>U</b> ser <b>I</b> nterface                             |
| <b>UI</b>    | <b>U</b> ser <b>I</b> nterface                                               |
| <b>ML</b>    | <b>M</b> achine <b>L</b> earning                                             |
| <b>ERD</b>   | <b>E</b> ntity <b>R</b> elationship <b>D</b> iagram                          |
| <b>GPS</b>   | <b>G</b> lobal <b>P</b> ositioning <b>S</b> ystem                            |
| <b>CI/CD</b> | <b>C</b> ontinuous <b>I</b> ntegration / <b>C</b> ontinuous <b>D</b> elivery |
| <b>JWT</b>   | <b>J</b> SON <b>W</b> eb <b>T</b> oken                                       |
| <b>UAT</b>   | <b>U</b> ser <b>A</b> cceptance <b>T</b> esting                              |

# Abstract

Lumpy Skin Disease (LSD) is a highly contagious viral infection in cattle that leads to severe economic losses and animal suffering, especially in rural regions where veterinary services are scarce. This project introduces **VDerm-X**, a mobile application that harnesses the power of Artificial Intelligence (AI), deep learning, and Natural Language Processing (NLP) to detect and manage LSD.

VDerm-X uses the MobileNetV2 model for high-accuracy, image-based diagnosis of lumpy skin disease, integrated with a chatbot that provides real-time consultation on symptoms, prevention, and treatment. The system features a user feedback mechanism for continuous model enhancement, a disease outbreak map to track and visualize reported cases, and real-time veterinary appointment booking for faster intervention. VDerm-X empowers livestock owners, supports veterinarians, and contributes to better animal health outcomes through scalable, tech-driven solutions.

# Chapter 1

## Introduction

### 1.1 Project Background

#### 1.1.1 Introduction

A vital component of agriculture, livestock production greatly boosts the economies and food security of many nations, including Pakistan. Nonetheless, the industry continues to encounter difficulties, most notably the spread of illnesses like cattle's Lumpy Skin Disease (LSD). In addition to endangering the health of the animals, these illnesses have a significant financial impact on farmers and the agricultural sector as a whole.

#### 1.1.2 The Challenge of Lumpy Skin Disease

Cattle that have Lumpy Skin Disease, a viral illness, develop skin nodules, fever, and decreased milk output. Containment is challenging since the disease spreads quickly through vectors like ticks and mosquitoes. LSD outbreaks have caused large financial losses and negatively impacted the livelihoods of innumerable farmers in areas like Punjab, where animal rearing is common.

#### 1.1.3 Limitations of Traditional Detection Methods

Laboratory testing and veterinary physical examinations are the traditional approaches used to identify LSD. Despite their effectiveness, these methods are frequently expensive, time-consuming, and unavailable to farmers in isolated locations. Widespread epidemics brought on by a delayed diagnosis could worsen the effects on the cattle sector.

### **1.1.4 The Role of Technology in Disease Detection**

New approaches to disease detection and treatment are made possible by technological advancements, especially in the areas of artificial intelligence (AI) and mobile applications. Cattle photos can be analyzed by AI-powered image recognition to find LSD symptoms, allowing for early detection and timely treatment. This procedure can be aided by mobile applications, which give farmers easily available resources to keep an eye on the wellbeing of their animals.

### **1.1.5 The VDerm-X Initiative**

The VDerm-X project intends to create a mobile application that uses AI for early illness identification in response to the difficulties presented by LSD and the shortcomings of conventional detection techniques. Through the program, users will be able to take pictures of their cattle, which an AI model trained to spot LSD symptoms would then assess. The app will also connect users with veterinary doctors for more help and offer treatment options.

### **1.1.6 Objectives of the Project**

The primary objectives of the VDerm-X project are:

- The goal is to create an easy-to-use mobile application for the early identification of LSD in cattle.
- To incorporate an artificial intelligence model that can examine pictures for indications of the illness.
- Recommendations for treatment are based on the AI analysis.
- To make it easier for farmers and veterinary professionals to communicate.
- Role-based access and a secure authentication system should be implemented.
- In order to improve monitoring and response, illness outbreaks should be geographically mapped and marked.

### **1.1.7 Significance of the Project**

There is a lot of promise for the VDerm-X project to enhance livestock health management in LSD-affected areas. The program can lessen the spread of the disease, lower financial losses, and increase the cattle industry's overall production by giving farmers easily available tools for early disease diagnosis. Additionally, updating agricultural techniques and adopting digital solutions for persistent problems are made possible by the combination of AI and mobile technologies.

### **1.1.8 Why We Choose**

The livestock business faces a serious problem with the incidence of lumpy skin disease in cattle, especially in areas like Punjab, Pakistan. Despite their effectiveness, traditional detection techniques are sometimes out of reach for many farmers, which causes delays in diagnosis and large epidemics. By creating a mobile application that uses AI for early disease diagnosis, the VDerm-X project aims to address this problem and give farmers a useful and effective tool to protect the health of their cattle.

## **1.2 Problem Statement**

### **1.2.1 Contextual Background**

A new infectious condition called Lumpy Skin condition (LSD) affects cattle and is typified by skin nodules, fever, and decreased milk output. The Lumpy Skin Disease Virus (LSDV), which belongs to the Capripoxvirus genus, is the cause of the illness. Since its discovery in Zambia in 1929, LSD has spread to Pakistan and other locations in Asia, the Middle East, and Africa. The illness presents serious problems for the health of animals and the agricultural sector, especially in areas where raising cattle is the main source of income.

### **1.2.2 Identification of the Problem**

The LSD outbreak in Pakistan has endangered farmers' livelihoods and caused significant financial losses. Farmers in distant places frequently lack access to laboratory tests and physical examinations, which are the foundation of traditional illness detection approaches. The disease is frequently discovered at an advanced stage due to a lack of early detection techniques, which reduces the effectiveness of therapy and raises the possibility of extensive epidemics.

### **1.2.3 Significance of the Problem**

The ramifications of not being able to identify LSD early are extensive. Cattle that are infected are less productive, and the disease may result in trade restrictions that have an impact on the whole economy. These problems are made worse by the dearth of easily available diagnostic instruments, especially for small-scale farmers who do not have the funds to get veterinary help right once. Resolving this issue is essential to protecting animal health and maintaining the cattle industry's viability.

### 1.2.4 Consequences of the Problem

The delayed detection and management of LSD result in:

- **Increased Mortality Rates:** Without early intervention, the disease can lead to higher death rates among cattle.
- **Economic Losses:** Farmers face financial hardships due to decreased milk production and the loss of livestock.
- **Spread of Disease:** The lack of early detection allows the disease to spread rapidly, affecting larger populations of cattle.

### 1.2.5 Existing Approaches and Limitations

Current approaches to LSD detection involve clinical diagnosis and laboratory confirmation. However, these methods have limitations:

- **Accessibility:** Laboratory facilities are often located far from rural farming communities.
- **Cost:** Diagnostic tests can be expensive, deterring farmers from seeking timely diagnosis.
- **Time-Consuming:** The process of obtaining and analyzing samples can delay diagnosis and treatment.

### 1.2.6 Research Gap

Tools that are easy to use, effective, and precise are obviously needed for the early identification of LSD. Although there are some technical solutions available, they are frequently not adapted to the requirements of farmers in developing nations. Research on creating smartphone apps that use artificial intelligence to offer real-time, approachable LSD diagnosis tools is lacking.

### 1.2.7 Problem Statement Summary

The main obstacle is the lack of easily available, effective, and precise instruments for the early identification of LSD in cattle. Farmers frequently lack the tools and knowledge necessary to detect the disease in its early stages, which causes treatment delays and higher death rates, particularly in rural areas. Furthermore, there is a lack of prompt, trustworthy therapy recommendations that are adapted

to the particular circumstances of the afflicted animals. The creation of creative solutions that integrate technology developments with the pragmatic requirements of farmers is necessary to address this problem.

## **1.3 Objectives**

### **1.3.1 Development of an AI-Powered Mobile Application for LSD Detection**

The main goal is to build and create **VDerm-X**, an intuitive mobile application that uses artificial intelligence (AI) to help diagnose lumpy skin disease in cattle early. The tool seeks to recognize distinctive lesions and symptoms linked to LSD from farmer-taken photos by applying sophisticated image processing techniques. The purpose of this tool is to enable livestock owners, especially those in isolated and underserved areas, to quickly identify disease symptoms, allowing for immediate intervention and limiting the spread of infection.

### **1.3.2 Integration of an AI-Driven Chatbot for Treatment Recommendations**

The program will include an AI-driven chatbot that will offer prompt, evidence-based therapy recommendations after LSD discovery, enhancing VDerm-X's diagnostic capabilities. By providing users with easily available advice on suitable therapeutic approaches, this feature seeks to close the gap between disease identification and management. To guarantee precise and context-specific guidance, the chatbot will be trained on an extensive dataset of veterinary knowledge. This will improve farmers' decision-making and lessen their need on urgent veterinary consultations.

### **1.3.3 Implementation of a Secure Authentication System with Role-Based Access Control**

It is crucial to guarantee the integrity and security of user data. Role-based access control (RBAC) will be incorporated into VDerm-X's strong authentication system to distinguish between veterinary professionals and other users, such as farmers. According to user roles, this system will limit access to particular features, protect sensitive data, and uphold data protection requirements. The program will help create a safe environment that supports stakeholder collaboration and data privacy by clearly defining user permissions.

### **1.3.4 Incorporation of Geolocation Features for Disease Outbreak Mapping**

In order to improve disease monitoring and control, VDerm-X will have geolocation features that allow LSD outbreak mapping. The application will produce real-time visualizations of illness spread patterns by gathering and examining geographic data linked to reported cases. This function will help policymakers and veterinary authorities keep an eye on the spread of LSD, allocate resources effectively, and put focused control measures in place. It would also encourage proactive health management measures by educating farmers on the incidence of diseases in their area.

### **1.3.5 Enhancement of User Experience through Intuitive Interface Design**

VDerm-X will give top priority to creating an intuitive and accessible user interface in recognition of the wide range of technological skills of potential consumers. To accommodate users with different degrees of digital literacy, the program will provide interactive tutorials, multilingual support, and clear navigation pathways. The application seeks to increase engagement, provide smooth interaction, and guarantee that all users can easily understand its functionalities by concentrating on user-centric design concepts.

### **1.3.6 Establishment of a Feedback Mechanism for Continuous Improvement**

VDerm-X will have a feedback system that enables users to voice concerns, recommend improvements, and exchange experiences in order to promote continuous improvement and response to user demands. The development team will be able to conduct iterative upgrades, discover areas for improvement, and obtain insightful data about customer satisfaction thanks to this functionality. The program will adapt to the real-world needs and expectations of its user base by keeping the lines of communication open for user input.

### **1.3.7 Promotion of Collaborative Networks among Stakeholders**

VDerm-X seeks to act as a platform that makes it easier for farmers, veterinarians, researchers, and legislators to work together. The tool will help create a unified network devoted to cattle health by facilitating communication, data exchange, and coordinated reactions to LSD outbreaks. It is expected that this cooperative

approach will increase the effectiveness of disease control tactics, foster knowledge sharing, and fortify the livestock industry's overall resilience.

## 1.4 Scope and Limitations

### 1.4.1 Scope of the Project

The development of the **VDerm-X** mobile application encompasses several key functionalities aimed at enhancing the early detection and management of Lumpy Skin Disease (LSD) in cattle:

- **Cross-Platform Mobile Application Development:** Utilizing React Native to guarantee compatibility with multiple mobile operating systems, allowing consumers with diverse devices to adopt it widely.
- **Machine Learning Integration for Image Analysis:** putting into practice a machine learning model designed to recognize LSD symptoms from photos of cattle, allowing for quick and precise disease detection.
- **AI-Powered Virtual Counselor for Therapy Advice:** integrating an intelligent chatbot to help farmers properly manage afflicted cattle by offering prompt, evidence-based treatment recommendations after disease discovery.
- **OTP-Verified Role-Based Access Control:** putting in place a reliable authentication mechanism that distinguishes between veterinary experts and regular users, guarantees proper access to program functionalities, and protects private information.
- **Geolocation Characteristics for Mapping Disease Outbreak:** mapping reported LSD instances using geolocation capabilities to help track the disease's progress and enable prompt action by the appropriate authorities.

### 1.4.2 Limitations of the Project

Although the **VDerm-X** application strives to offer complete assistance for LSD identification and control, some restrictions are recognized:

- **Relative to Image Quality:** The quality of the photographs that people upload determines how accurately diseases are detected. Inappropriate framing, low quality, or poor lighting can all make it more difficult for the machine learning model to recognize symptoms.

- **Recommendations for Chatbots:** Because the AI chatbot's treatment recommendations are predicated on pre-established datasets, they might not cover every scenario or take special circumstances into consideration, hence in complex situations, professional veterinary assistance is required.
- **Requirements for Internet connectivity:** Some of the application's capabilities, like access to updated treatment protocols and real-time data synchronization, depend on reliable internet connectivity, which isn't always available in remote locations.
- **User Adoption and Training:** Users' desire to embrace new digital tools and their familiarity with smartphone technology, which may range among various groups, are key factors in the application's effective use.
- **Data Security and Privacy Issues:** To prevent unwanted access and guarantee adherence to data protection laws, handling sensitive user and livestock data requires strong security measures.

## 1.5 Methodology Overview

The development of **VDerm-X** follows an Agile methodology, promoting iterative development and continuous feedback. The process involves:

- **Requirement Analysis:** Determining the requirements of veterinarians and farmers.
- **Design:** Developing user-friendly system architecture and interfaces.
- **Implementation:** Using NestJS for the backend and React Native for the frontend, the application was developed.
- **Testing:** Carrying out integration and unit tests to guarantee dependability and functioning.
- **Deployment:** Allowing the intended audience to utilize the program.

### 1.5.1 Research Design

This study employed a quantitative, applied research design to develop VDerm-X, a mobile application integrating AI technologies for the early detection of animal skin diseases. The project followed agile development principles, facilitating iterative enhancements based on continuous feedback.

### 1.5.2 Requirement Analysis

An initial needs assessment identified a gap in accessible tools for early detection of animal skin diseases. The primary objectives were to develop a user-friendly mobile application and define two distinct user roles: General Users (pet owners) and Veterinary Professionals (requiring admin approval).

### 1.5.3 Core Functionalities

- Image-based disease prediction
- AI-driven treatment recommendations
- Secure authentication with OTP verification
- Admin-controlled onboarding for veterinary professionals
- Breakout location marking on map using geo-coordinates

### 1.5.4 Technology Stack

- **Frontend:** React Native (Expo) for cross-platform mobile development
- **Backend:** NestJS for API development and business logic
- **Database:** MongoDB for storing user data, history, and roles
- **Machine Learning:** TensorFlow using MobileNetV2 architecture
- **AI Chatbot:** OpenAI GPT model fine-tuned with veterinary-specific data

### 1.5.5 Machine Learning Model Evaluation

Several Convolutional Neural Network (CNN) architectures were evaluated. MobileNetV2 was selected due to its high accuracy (92%), lightweight structure suitable for mobile devices, and faster inference times compared to larger models.

### 1.5.6 Model Training and Deployment

The MobileNetV2 model was trained on a labeled dataset comprising images of various animal skin diseases. Post-training, the model was exported in .h5 format and served using the FastAPI framework. A secure API endpoint was established to facilitate real-time predictions within the application.

### 1.5.7 AI Chatbot Integration

An AI chatbot powered by OpenAI's GPT model was integrated to provide users with concise and relevant treatment advice. The model was fine-tuned using domain-specific veterinary data to ensure accuracy. The chatbot becomes accessible within the app's chat interface following a disease detection event.

### 1.5.8 Authentication and Role Management

- **OTP Authentication:** Implemented via email or phone number verification during user registration
- **Admin Approval Workflow:** Registrations as veterinary professionals are flagged for admin review. Approved users gain access to extended features within the app

### 1.5.9 Frontend Development

The frontend was developed using React Native (Expo), featuring screens for registration, OTP verification, image upload, prediction results, chatbot interaction, and the veterinary dashboard for approved professionals. Geo-marking is enabled to identify locations of disease outbreaks. React Navigation was used for screen routing.

### 1.5.10 Backend Development

Backend APIs were developed using NestJS to handle image uploads, prediction requests, chatbot queries, and user management. MongoDB was integrated for data persistence, with role-based access control ensuring appropriate feature accessibility.

### 1.5.11 Testing and Optimization

API testing was conducted using Postman. Optimizations included reducing image size for faster uploads and implementing HTTPS, token-based authentication, and role-based access control for secure access.

### 1.5.12 Future Enhancements

- Expand support to multiple diseases and animal species
- Introduce an analytics dashboard for veterinary professionals and administrators
- Incorporate TensorFlow Lite for offline predictions

- Enhance the chatbot to include preventive care suggestions
- Add push notifications for follow-up reminders

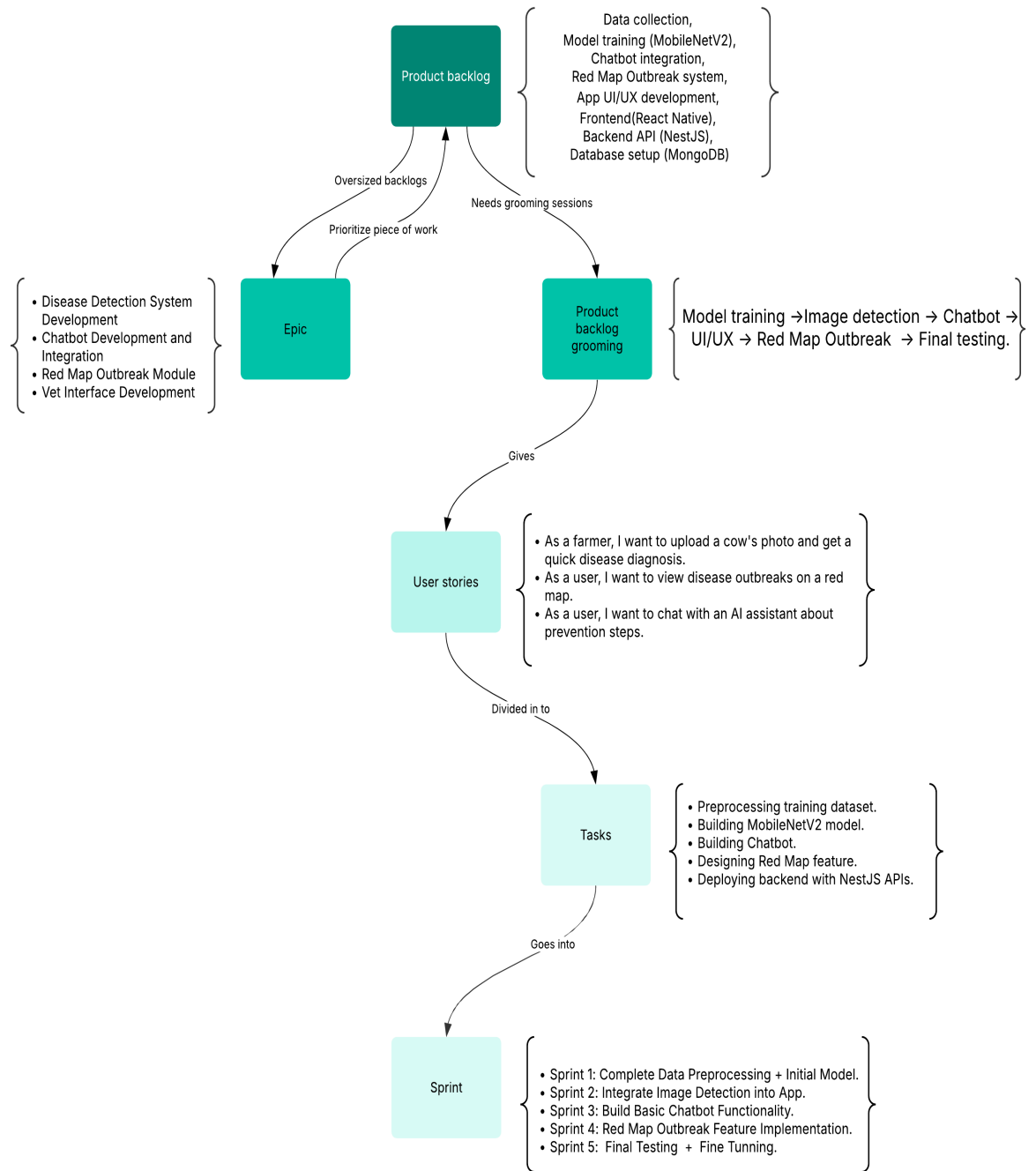


FIGURE 1.1: Methodology Diagram

## 1.6 Report Organization

This thesis is structured as follows:

- **Chapter 1: Introduction** – Provides background information, problem statement, objectives, scope, and methodology.
- **Chapter 2: Software Requirements Specification (SRS)** – Details the functional and non-functional requirements of the system.
- **Chapter 3: System Design** – Discusses the architectural design, including diagrams and data flow.
- **Chapter 4: Implementation** – Describes the development process, tools used, and key implementation details.
- **Chapter 5: Testing** – Covers the testing strategies, test cases, and results.
- **Chapter 6: Deployment and User Manual** – Provides deployment steps and user guidance.
- **Chapter 7: Results, Evaluation, and Conclusion** – Summarizes the outcomes, evaluates the system, and suggests future enhancements.

# Chapter 2

## Software Requirements Specification (SRS)

### 2.1 Software Requirements Specification (SRS)

#### 2.1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) document is to delineate the functional and non-functional requirements for the development of the **VDerm-X** mobile application. This application is intended to enable the early detection and management of Lumpy Skin Disease (LSD) in cattle by integrating artificial intelligence (AI) technologies. By presenting an overall overview of the system's proposed functionalities, performance parameters, and constraints, this document provides a reference basis for all stakeholders engaged in the project lifecycle, including developers, testers, project managers, and end-users.

#### 2.1.2 Intended Audience

This SRS report is meant for a wide range of stakeholders, each with individual interests and tasks related to the VDerm-X application:

- **Project Managers and Developers:** In order to grasp the in-depth requirements and have proper implementation of the functionalities of the system.
- **Veterinary Professionals:** To provide domain-specific insights that inform the AI model's development and the chatbot's treatment recommendations.
- **End-Users (Farmers and Livestock Owners):** To utilize the application for early detection of LSD and receive timely treatment guidance.

- **Regulatory Bodies and Policy Makers:** To assess the application's compliance with relevant standards and its potential impact on animal health management practices.

### 2.1.3 Definitions and Acronyms

To ensure clarity and a common understanding among all stakeholders, the following definitions and acronyms are used throughout this document:

- **AI (Artificial Intelligence):** The simulation of human intelligence processes by machines, particularly computer systems, to perform tasks such as learning, reasoning, and self-correction.
- **API (Application Programming Interface):** A set of protocols and tools that allow different software applications to communicate with each other.
- **GUI (Graphical User Interface):** A user interface that includes graphical elements, such as windows, icons, and buttons, allowing users to interact with electronic devices.
- **LSD (Lumpy Skin Disease):** A viral disease affecting cattle, characterized by nodules on the skin and other symptoms, leading to significant economic losses in the livestock industry.
- **ML (Machine Learning):** A subset of AI that enables systems to learn and improve from experience without being explicitly programmed.
- **OTP (One-Time Password):** A security mechanism that generates a unique password for a single transaction or login session, enhancing authentication processes.
- **RBAC (Role-Based Access Control):** A method of regulating access to computer or network resources based on the roles of individual users within an organization.
- **SRS (Software Requirements Specification):** A comprehensive description of the intended purpose and environment for software under development, detailing functional and non-functional requirements.
- **VDerm-X:** The proposed mobile application designed to detect and manage Lumpy Skin Disease in cattle using AI technologies.

## 2.2 Overall Description

### 2.2.1 Product Perspective

The **VDerm-X** application is a standalone mobile solution designed to facilitate the early detection and management of Lumpy Skin Disease (LSD) in cattle through the integration of artificial intelligence (AI) technologies. It operates independently but may interface with external systems for functionalities such as geolocation mapping and data storage.

#### System Interfaces:

- **Geolocation Services:** Uses device GPS to map disease outbreaks.
- **Cloud Storage:** Interfaces with cloud databases for storing user data and disease reports.

#### User Interfaces:

- **Mobile Application UI:** Developed with a user-friendly interface for ease of use by farmers and veterinary professionals.
- **Chatbot Interface:** Offers interactive communication for treatment suggestions.

#### Hardware Interfaces:

- **Mobile Devices:** Supports Android and iOS smartphones with cameras and GPS.

#### Software Interfaces:

- **Operating Systems:** Compatible with Android and iOS operating systems.
- **Third-Party APIs:** Supports APIs for features like OTP verification and map services.

#### Communications Interfaces:

- **Internet Connectivity:** Needs internet connectivity for features like data synchronization and chatbot interactions.

**Memory Constraints:**

- **Storage Requirements:** Limited storage required for application installation; extra storage to store image data and offline capabilities.

**Operations:**

- **User Operations:** End users can take actions .
- **Administrative Operations:** Veterinarians can administer treatments and monitor disease outbreaks.

**Site Adaptation Requirements:**

- **Localization:** Application has support for multiple languages for different user bases.
- **Customization:** Adjustable settings to conform to local disease management protocols

## 2.2.2 User Characteristics

**Farmers and Livestock Owners:**

- **Technical Expertise:** Differing degrees; software developed for simplicity of use.
- **Responsibilities:** Take photos of cattle, read AI diagnoses, and apply suggested treatments.

**Veterinary Professionals:**

- **Technical Expertise:** High; well-versed in disease management procedures.
- **Responsibilities:** Offer expert supervision, authenticate AI diagnoses, and revise treatment protocols.

**Regulatory Authorities:**

- **Technical Expertise:** Moderate; oversee disease outbreak data and implement control measures.
- **Responsibilities:** Monitor disease trends, allocate resources, and enforce health regulations.

### 2.2.3 Assumptions and Dependencies

- **Device Availability:** Users possess smartphones with camera and GPS capabilities.
- **Internet Access:** Necessary for data synchronization and accessing cloud-based services.
- **User Compliance:** Users will follow application guidelines for capturing images and implementing treatments.
- **Data Privacy Regulations:** Application complies with regional data protection laws.
- **Third-Party Services:** Continued availability and reliability of integrated APIs and services.

## 2.3 Specific Requirements

This section delineates the specific requirements of the VDerm-X application, encompassing functional, non-functional, and interface requirements. These requirements are structured to provide clarity and facilitate the development, testing, and validation processes.

### 2.3.1 Functional Requirements

The functional requirements define the core functionalities that the VDerm-X application must perform to meet its objectives.

#### 2.3.1.1 User Registration and Authentication

- The system shall allow users to register using a valid mobile number and receive a One-Time Password (OTP) for verification.
- The system shall differentiate between general users and veterinary professionals during the registration process.
- The system shall implement Role-Based Access Control (RBAC) to grant appropriate access levels based on user roles.

#### 2.3.1.2 Image Capture and Analysis

- The system will allow users to take or upload photos of cattle skin lesions.
- The system will analyze the images using a pre-trained Machine Learning (ML) model to identify indications of Lumpy Skin Disease (LSD).

- The system will present the analysis results, showing the probability of LSD presence.

#### **2.3.1.3 Chatbot for Treatment Recommendations**

- The system will offer an AI-driven chatbot to provide treatment suggestions based on the results of analysis.
- The chatbot will be able to respond to user questions about LSD symptoms and treatment.

#### **2.3.1.4 Geolocation and Disease Mapping**

- The system will use the device's GPS to get the location of the user on consent.
- The system will plot reported LSD cases to see disease outbreaks graphically.
- The system will enable veterinary experts to see and analyze patterns of disease spread.

#### **2.3.1.5 Notifications and Alerts**

- The system will provide users with notifications about disease outbreaks in their area.
- The system will notify users of updated treatment protocols or news.

#### **2.3.1.6 Data Management**

- The system will securely store user information in accordance with data protection policies.
- The system will permit users to view and maintain their submitted reports and analysis history.

### **2.3.2 Non-Functional Requirements**

The non-functional requirements identify the quality attributes and constraints of the VDerm-X application.

#### **2.3.2.1 Usability**

- The application should have an easy-to-use and intuitive interface appropriate for users with mixed technical proficiency.
- The application should be multi-language capable to address a diverse group of users.

#### **2.3.2.2 Performance**

- The system should take less than 5 seconds to process and analyze images under normal network conditions.
- The system should be capable of supporting up to 10,000 concurrent users without deterioration in performance.

#### **2.3.2.3 Security**

- The system shall encrypt all user data at the time of transmission and storage.
- The system shall employ secure authentication measures, such as OTP validation and RBAC.

#### **2.3.2.4 Reliability**

- The system shall maintain 99.9% uptime to be available round-the-clock.
- The system shall resume operation within 2 minutes in case of failures to reduce downtime.

#### **2.3.2.5 Maintainability**

- The system architecture should be modular to allow for seamless updation and maintenance.
- The system shall offer detailed logs to support troubleshooting and issue resolution.

#### **2.3.2.6 Portability**

- The application should be cross-platform on both Android and iOS.
- The application should operate on devices with different screen resolutions and sizes.

### **2.3.3 Interface Requirements**

This section describes the interfaces between the VDerm-X application and external entities, i.e., users, hardware, and other computer programs.

### **2.3.3.1 User Interfaces**

- The application should offer an intuitive and responsive interface to the graphical user interface (GUI) for easy user usage.
- The chatbot interface should be capable of natural language processing to interpret and answer user questions well.

### **2.3.3.2 Hardware Interfaces**

- The application will leverage the camera on the device for taking photographs of cattle skin lesions.
- The application will tap the device's GPS module for reading location data to map disease.

### **2.3.3.3 Software Interfaces**

- The application will include third-party APIs for functionalities like OTP validation and map operations.
- The application will talk to cloud storage offerings for safely storing and retrieving user data as well as results of analysis.
- The application shall be compatible with standard operating system services for notifications and background processing.
- The application shall utilize secure communication protocols (e.g., HTTPS) for all data transmissions to and from external services.

### **2.3.3.4 Communications Interfaces**

- The application shall communicate with external services over standard internet protocols, primarily HTTP/HTTPS.
- The application shall support real-time data synchronization with cloud services to ensure up-to-date information is available to users.
- The application shall handle intermittent connectivity gracefully, queuing data for transmission when the connection is re-established.
- The application shall comply with relevant communication standards and regulations to ensure interoperability and data security.

# Chapter 3

## System Design

Write in your own chapter title to set the page header

Following diagrams serve to solidify our approach to above-mentioned requirements, dependencies, and constraints. These diagrams include:

### 3.1 Architecture Diagram

The architecture of the VDERM-X system provides a high-level overview of its key components and their interactions. This diagram illustrates the core modules and the flow of data between them.

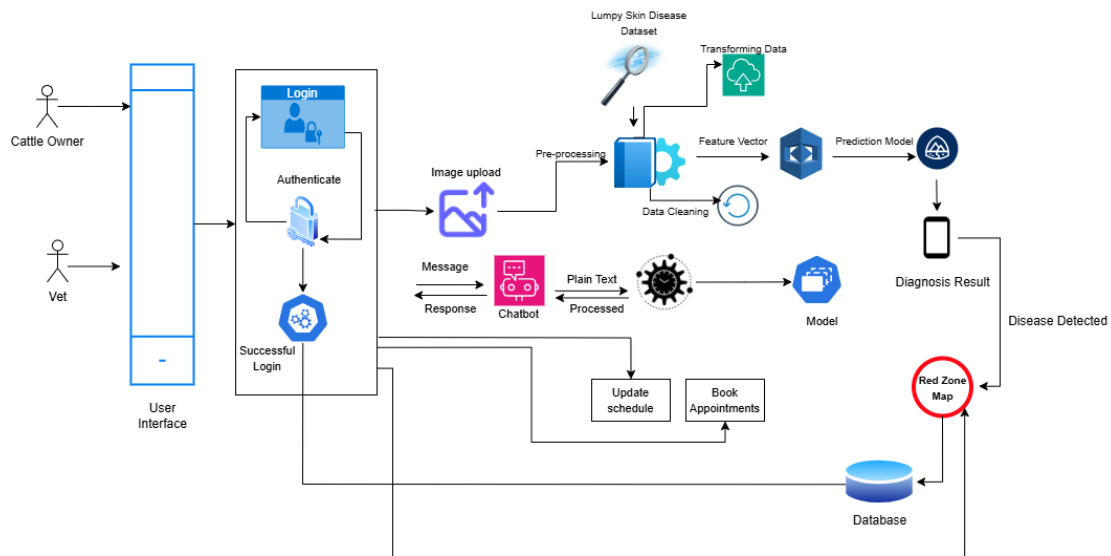


FIGURE 3.1: System Architecture Diagram of VDERM-X.

## 3.2 Use Case Diagram

A Use Case Diagram represents the functional requirements of VDERM-X by depicting the interactions between various users (actors) and the system.

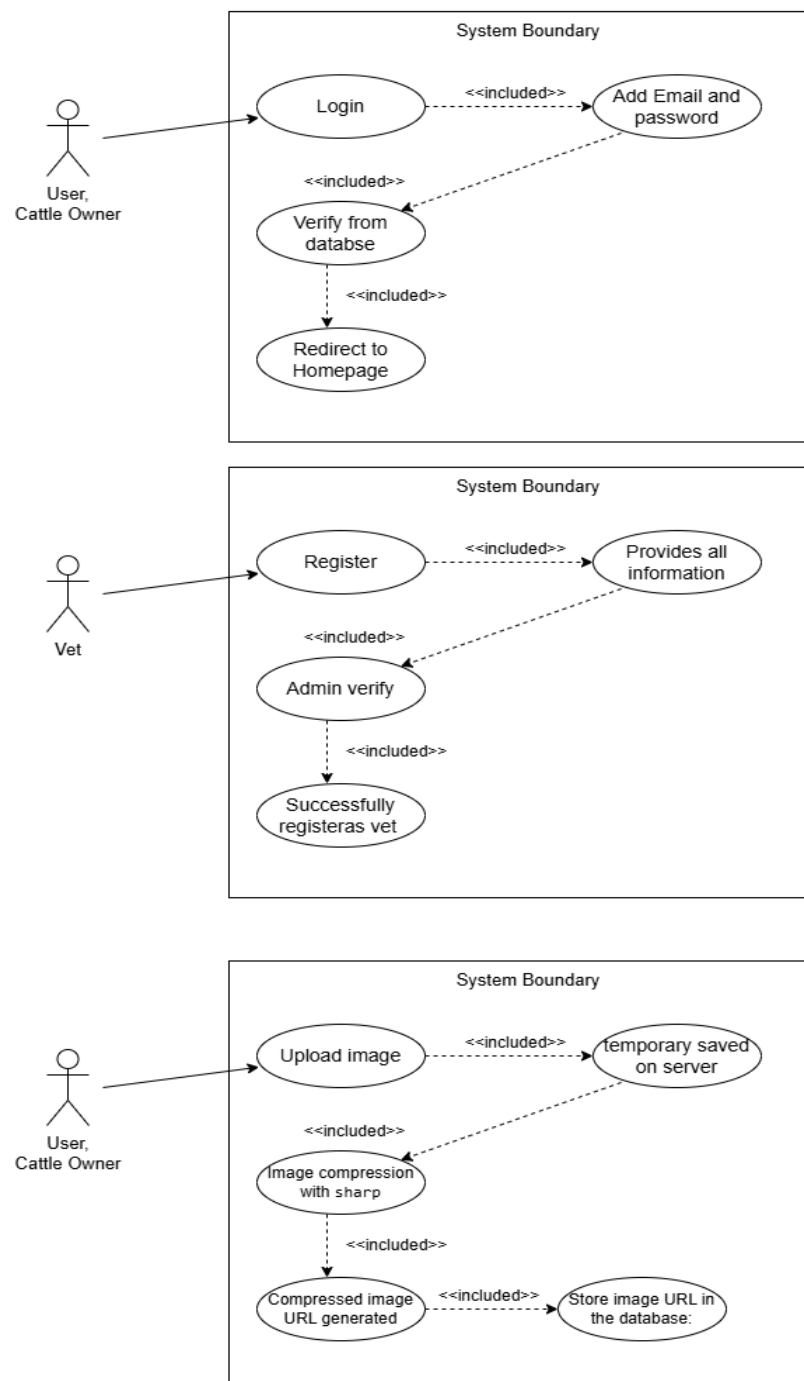


FIGURE 3.2: A Generic Use Case Diagram for VDERM-X Showing Authentication.

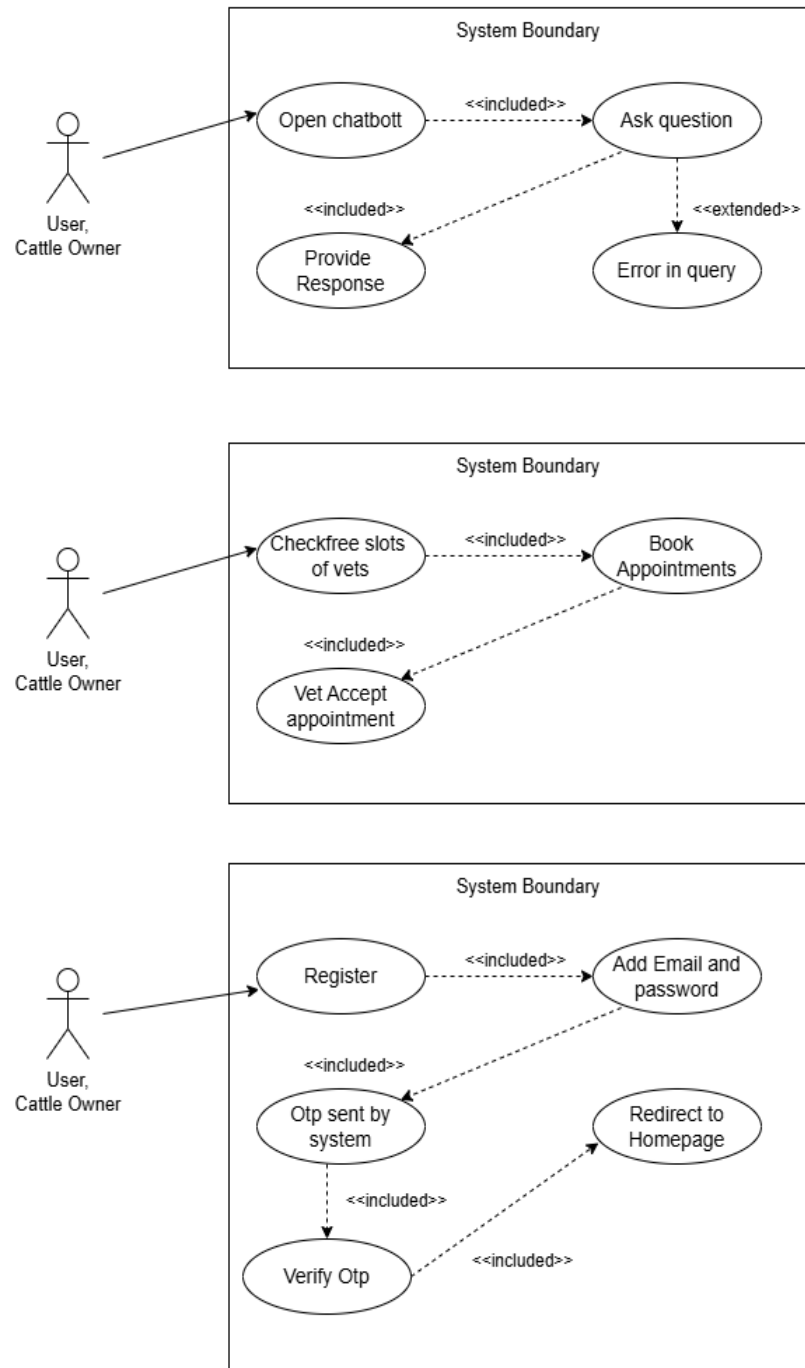


FIGURE 3.3: A Use Case Diagram for VDERM-X, Showing The Extended Interaction Between the System and Cattle Owners.

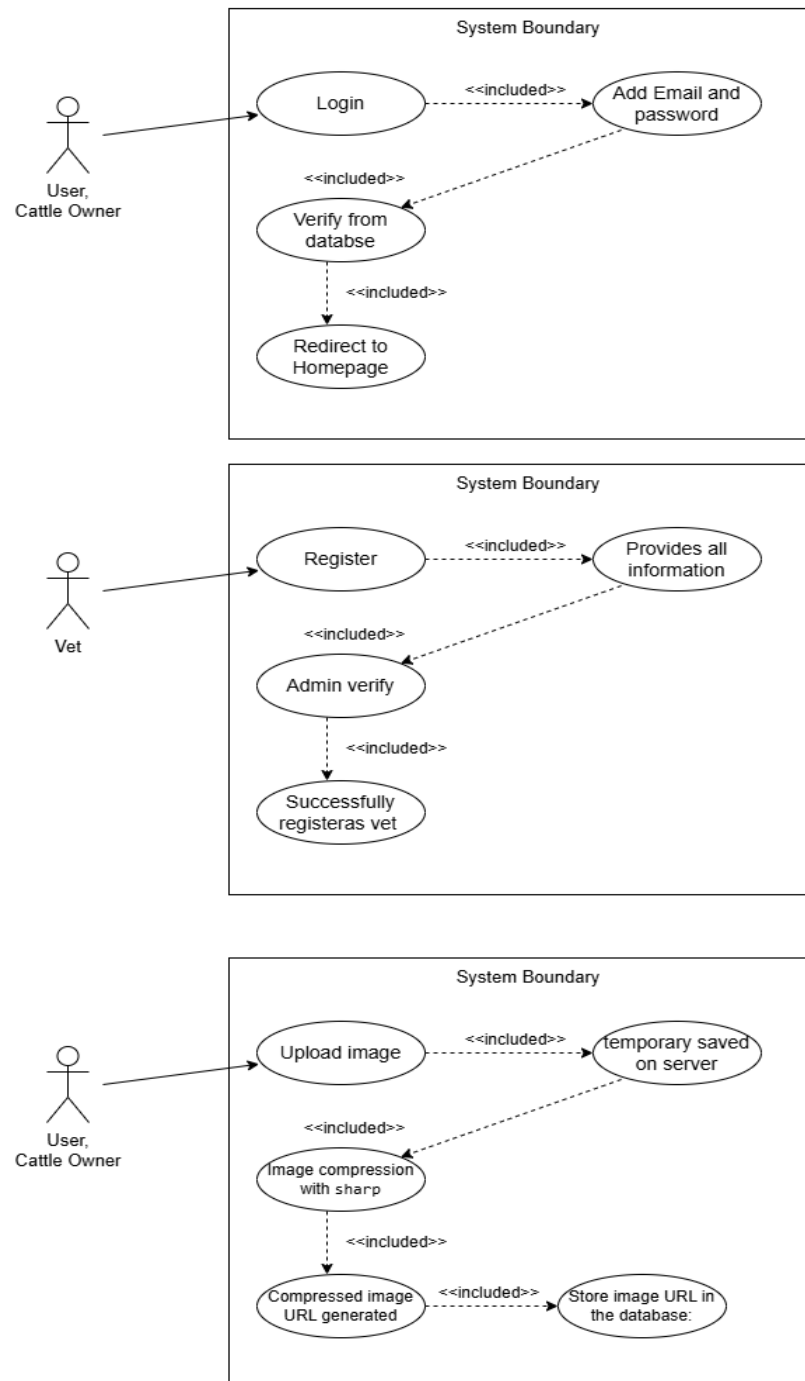


FIGURE 3.4: A Use Case Diagram for VDERM-X, Showing the Extended Interaction Between the System and its Actors.

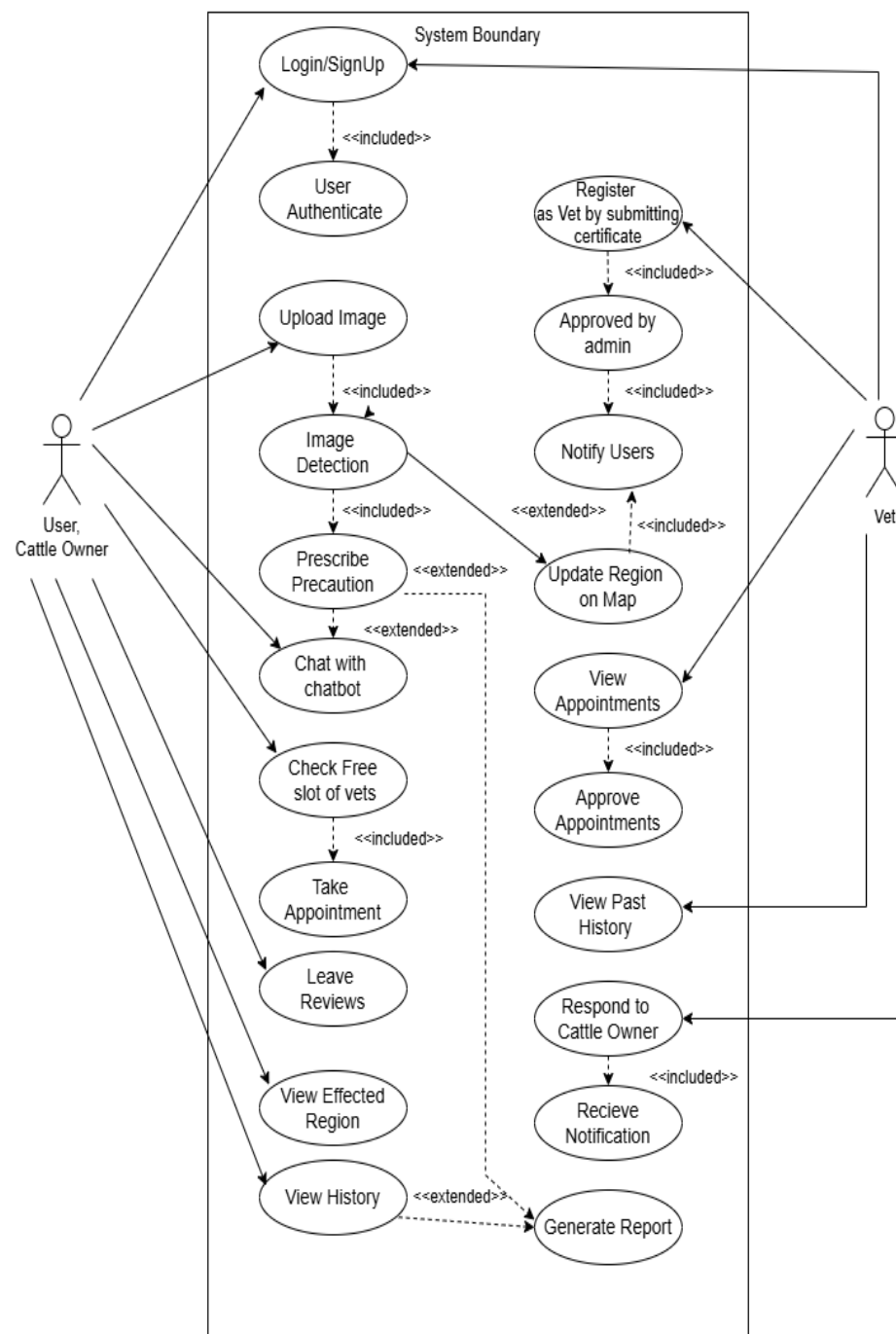


FIGURE 3.5: A Use Case Diagram for VDERM-X, Showing the Extended Interaction Between the System and its Actors.

### 3.3 Class Diagram

The Class Diagram offers a static representation of the system's architecture by illustrating its classes, their attributes, operations, and interrelationships. It helps in understanding how different parts of the system are connected and how they interact structurally.

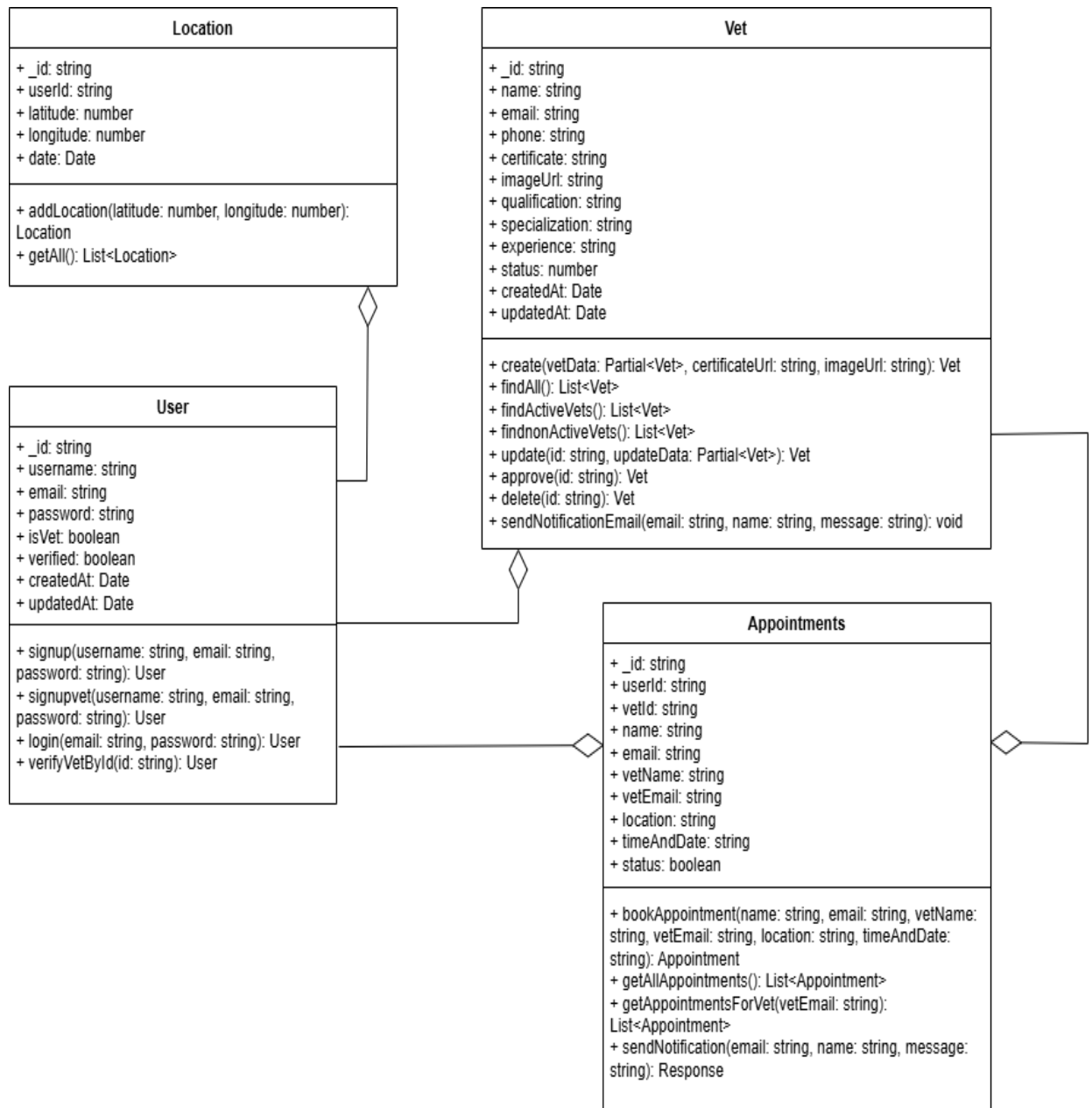


FIGURE 3.6: Class Diagram for VDERM-X, Modelling its Object-Oriented Structure.

### 3.4 Sequence Diagram

The Sequence Diagram of VDERM-X, illustrates the interactions between system components over time, focusing on the sequence of message passing for a specific business functionality.

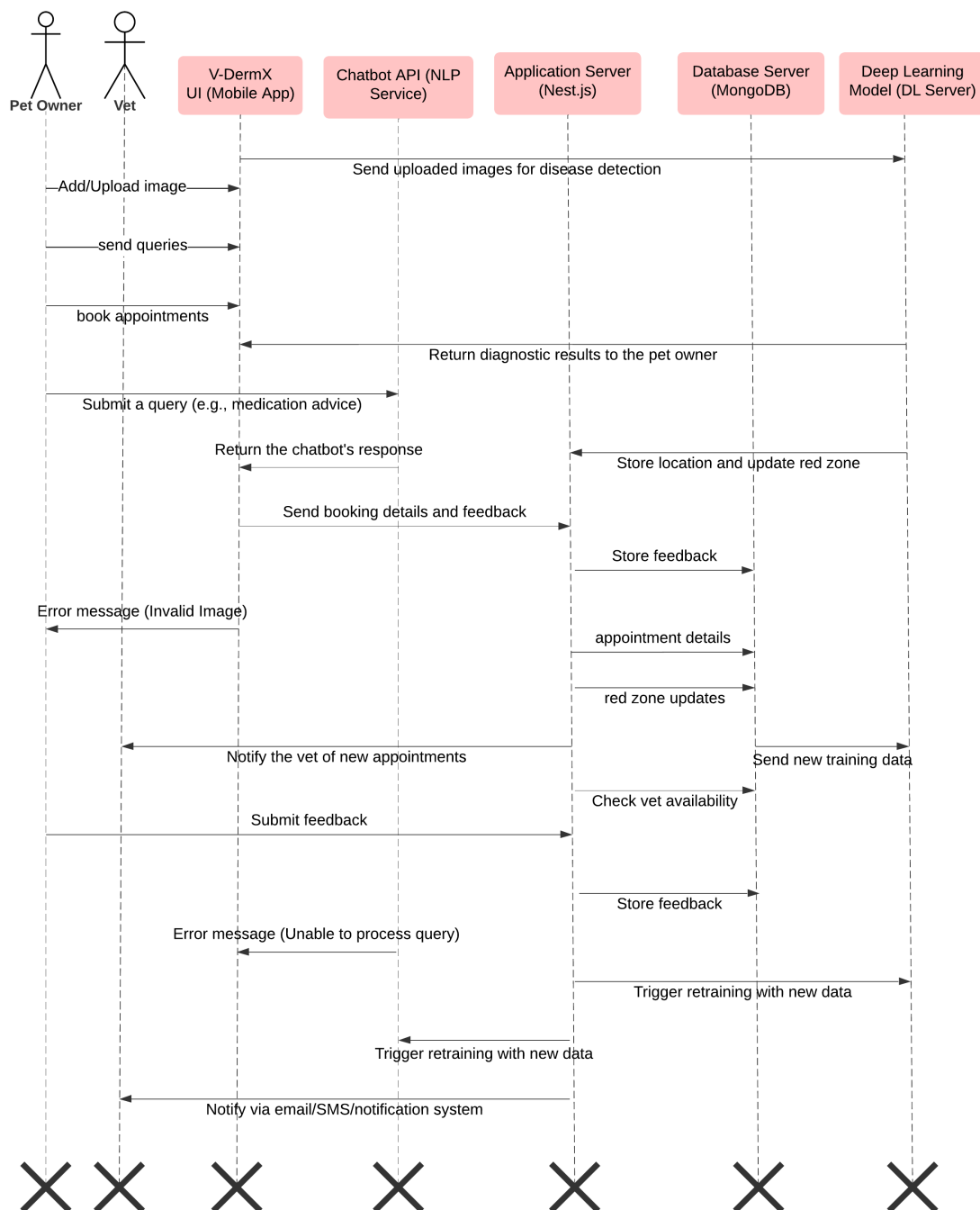


FIGURE 3.7: Sequence Diagram VDERM-X

## 3.5 UI Mockups

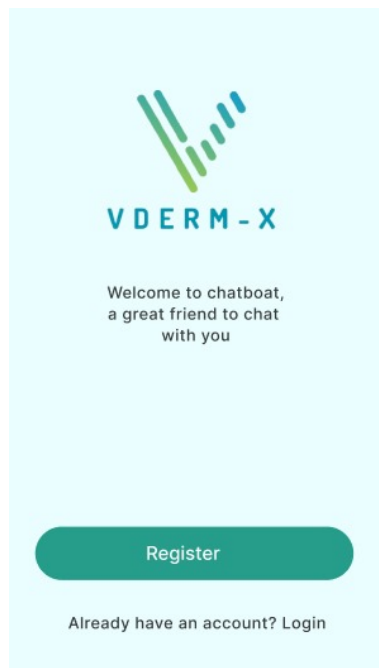


FIGURE 3.8: First Screen

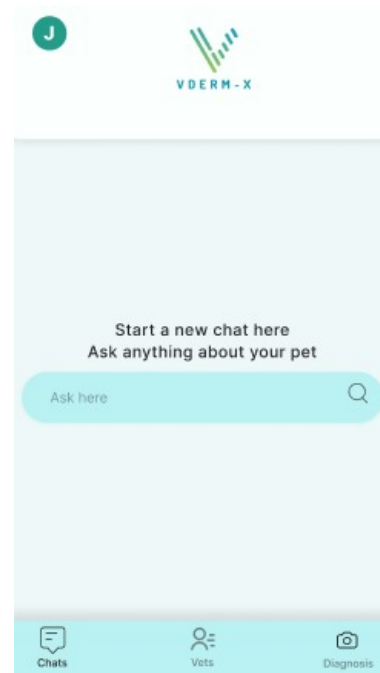


FIGURE 3.9: Chatbot Screen

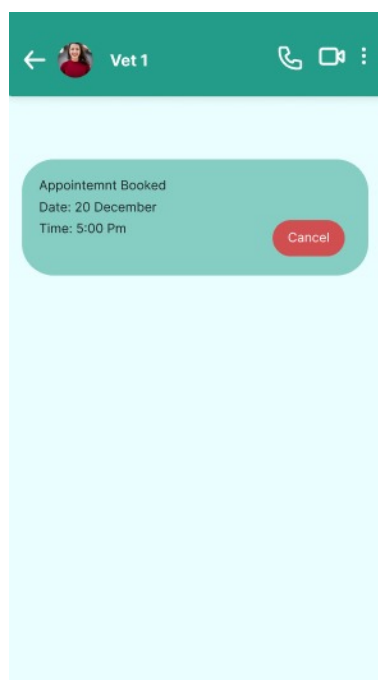


FIGURE 3.10: User Side Vet Appointment Screen

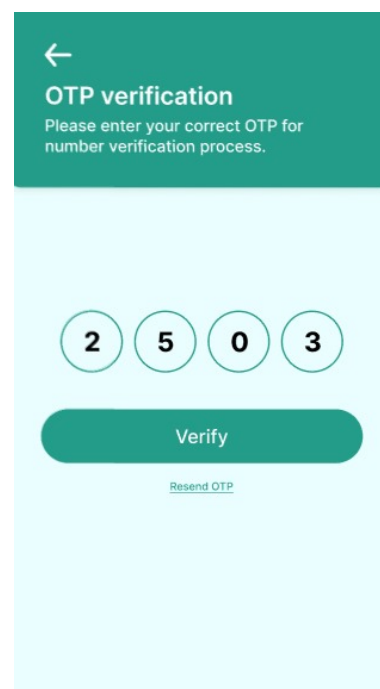


FIGURE 3.11: OTP Verification Screen

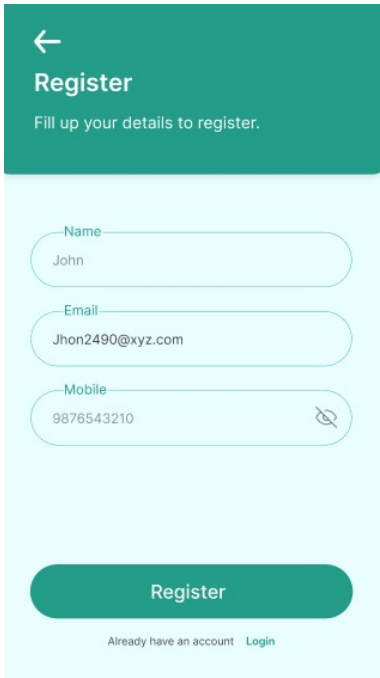


FIGURE 3.12: Register Screen

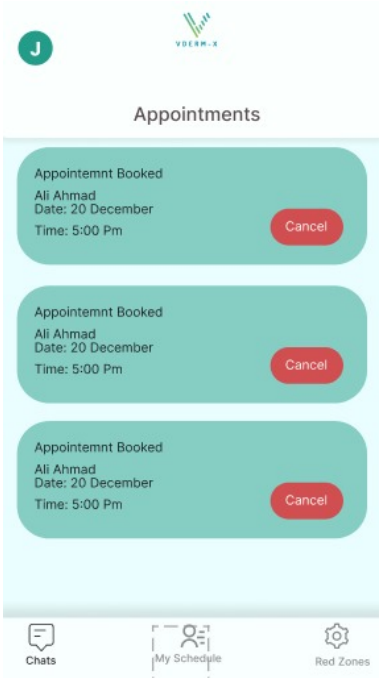


FIGURE 3.13: Vet Appointment Screen

## 3.6 Database Diagram

A Database Diagram illustrates the structure of the VDERM-X database by showcasing the relationships between different data entities and their attributes. It offers a visual representation of how data is organized, stored, and connected within the system. This diagram helps in understanding the logical flow of information, identifying primary keys, foreign keys, and constraints, and ensuring data integrity.

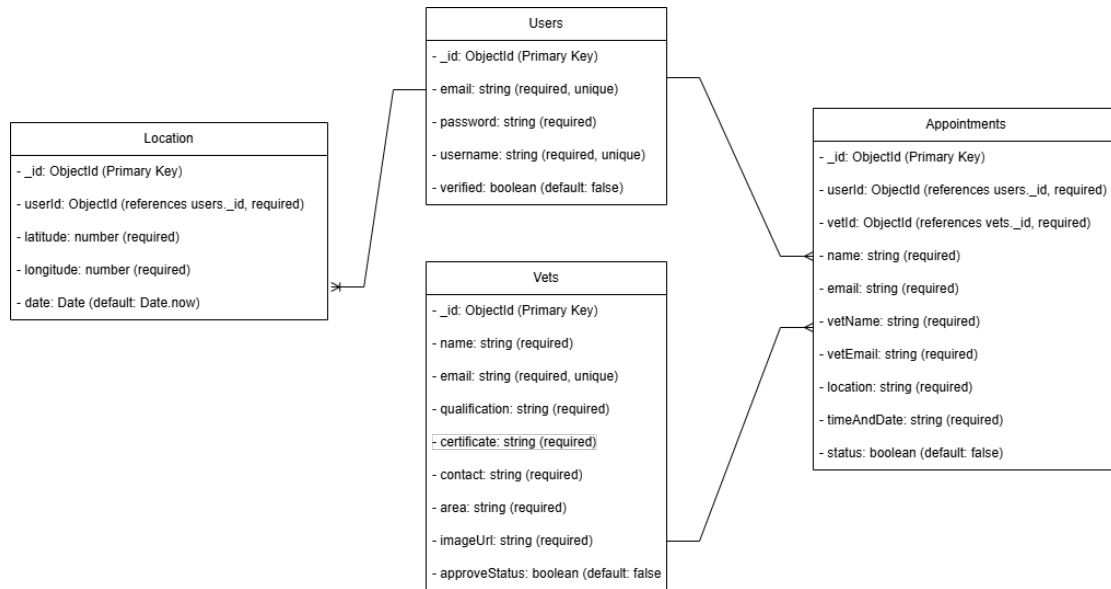


FIGURE 3.14: Database Diagram of VDERM-X.

# Chapter 4

## Implementation

This chapter describes the implementation of the VDerm-X system, including the tools and technologies used, the design and function of each module, key implementation strategies, and visual snapshots of the system in operation.

### 4.1 Development Tools & Technologies

The creation of **VDerm-X** utilized a strong and contemporary set of technologies to guarantee expandability, efficiency, and a focus on the user experience. Here is a detailed examination of the instruments and technologies implemented in different aspects of the application:

#### 4.1.1 Frontend Development

- **React Native with TypeScript:** The mobile app was built utilizing React Native, allowing it to work seamlessly on both Android and iOS devices. Incorporating TypeScript introduced static typing, which improved the dependability and upkeep of the code. This pairing allowed for swift development phases and a uniform user experience across different platforms.
- **Offline Functionality:** It can support users in regions with poor internet access, offline functionalities can be introduced through local storage options and caching methods. This can guarantee consistent availability of important features and information.
- **UI/UX Design:** The software utilized libraries of components along with personalized design elements to develop a user-friendly and adaptable interface. It emphasizes accessibility and integrating methods for user feedback significantly improved the complete experience for users.

### 4.1.2 Backend Development

- **Node.js with Express.js:** The first version of the backend was built using Node.js alongside Express.js, which offers an adaptable foundation for creating RESTful APIs and managing asynchronous tasks effectively.
- **Migration to NestJS:** As the needs of the application changes, the backend system moved to NestJS. This modern framework based on Node.js provided a component-based design, support for dependency injection, and built-in features for scalable microservices, enhancing the structure and upkeep of the code.
- **API Development and Security:** Comprehensive API endpoints were developed to handle various functionalities, including user authentication, data retrieval, and interactions with machine learning models. Security measures such as JWT-based authentications and input validations were implemented to protect user data and ensure secure communications.

### 4.1.3 Database Management

- **MongoDB:** A NoSQL database known as MongoDB was selected due to its adaptability in managing various forms of data, both structured and semi-structured. It effectively organizes collections for user profiles, reports on diagnostics, logs from chatbots, and feedback from the system. The absence of a fixed schema allows for quick changes throughout the development process.
- **Data Modeling and Indexing:** Collections were structured with adaptable schemas that progressed alongside application logic. Indexes were utilized to enhance read and write processes, particularly for user activity records and search parameters.

### 4.1.4 Machine Learning Integration

- **MobileNetV2 using TensorFlow and Keras:** A convolutional neural network from MobileNetV2 was utilized to train a model with TensorFlow and Keras in order to identify lumpy skin disease from images of cattle. The model was chosen due to its high accuracy, compact architecture, and mobile deployability.
- **Model Optimization:** After training, the model was quantized to minimize its size and improve inference speed on mobile. Pruning and float16

precision were among the techniques investigated to balance performance and efficiency.

- **Model Deployment:** The model was deployed as a RESTful service with FastAPI and integrated into the backend to enable real-time predictions from the mobile app.

#### 4.1.5 Chatbot System

- **AI-Powered Chatbot:** A domain-adapted chatbot was built using a fine-tuned language model trained on over 1000 veterinary questions. It provided users with symptom analysis, preventive advice, and basic treatment options.
- **REST API Integration:** The chatbot was exposed via REST APIs, enabling seamless communication between the mobile frontend and AI service. Responses were tailored based on prediction results from the image model.

#### 4.1.6 Geolocation and Mapping

- **Device GPS Integration:** With user consent, GPS data was collected and sent along with disease reports to enable location-based tracking.
- **Map Visualization:** The application used Mapbox and GeoJSON to display cases on interactive maps, with dynamic heatmaps to monitor outbreak patterns and spread over time.

#### 4.1.7 Veterinary Appointment System

- **Real-time Booking Module:** Users could schedule veterinary appointments based on real-time availability of approved vets. Bookings were synced with backend calendars and integrated with notification systems.
- **Push Notifications and Scheduling Logic:** Users received timely reminders and updates on upcoming appointments. Backend APIs managed conflict resolution and schedule overlaps.

#### 4.1.8 Version Control and CI/CD

- **GitHub and GitHub Actions:** GitHub was used for source control, with pull requests and branch management for collaboration. GitHub Actions performed automated linting and test checks on new commits.
- **Continuous Integration:** CI pipelines ensured that each deployment passed build and test stages before being merged into production.

### 4.1.9 Development Environment

- **VS Code and Dev Tooling:** Visual Studio Code served as the primary IDE with extensions for debugging, auto-formatting, and linting.
- **Model Training Platform:** Jupyter Notebook and Google Colab (with GPU runtime) were used for experimentation, model training, and evaluation. Datasets were versioned, and experiment results were tracked using notebook logs.

## 4.2 Description of Modules

The architecture of VDerm-X is designed around clearly defined modules, each responsible for a critical component of the application's functionality. The modular approach ensures maintainability, scalability, and separation of concerns throughout the development and operational lifecycle.

### 4.2.1 User Authentication Module

This module manages the registration and authentication of users, supporting both general users and veterinary professionals. It implements:

- OTP-based verification using email or mobile numbers.
- Secure login sessions with token-based authentication (JWT).
- Role-Based Access Control (RBAC) to determine available features per user type.
- Offline login support using encrypted locally stored session tokens.

It ensures secure identity verification, prevents unauthorized access, and enables a seamless login experience, even in rural or low-connectivity areas.

### 4.2.2 Disease Detection Module

This module is at the core of the VDerm-X system. It allows users to:

- Upload or capture real-time images of suspected lesions on animal skin.
- Automatically preprocess and resize images for model compatibility.
- Submit images to a trained MobileNetV2-based deep learning model.

- Receive predictions about the likelihood of Lumpy Skin Disease, along with a confidence score.

The model was trained on a curated dataset of cattle images, achieving high accuracy and optimized for mobile deployment using TensorFlow Lite.

### **4.2.3 Chatbot Assistance Module**

The chatbot module offers AI-driven support through:

- An intuitive chat interface embedded within the app.
- A response engine powered by a fine-tuned natural language model.
- Capability to answer questions about symptoms, treatments, and prevention of LSD and related skin diseases.
- Contextual support based on model predictions and user queries.

This module aims to provide immediate veterinary advice, helping users make informed decisions while waiting for a professional consult.

### **4.2.4 Outbreak Mapping and Reporting Module**

This geolocation-powered module supports public health surveillance by:

- Allowing users to report confirmed or suspected cases of LSD.
- Capturing and storing geocoordinates (with user permission) for each report.
- Visualizing outbreaks on an interactive map using libraries like Mapbox.
- Generating heatmaps to illustrate outbreak intensity and spread over time.

This data can be used by authorities to track, predict, and respond proactively to animal health crises.

### **4.2.5 Veterinary Appointment Booking Module**

This module simplifies access to certified veterinary support by:

- Listing available veterinary professionals verified through the admin portal.
- Displaying consultation availability through calendar scheduling.
- Allowing users to request virtual or physical appointments.
- Sending appointment confirmations and reminders via push notifications.

## 4.3 Key Implementation Details

The implementation of VDerm-X involved a series of technical considerations to optimize accuracy, performance, and user privacy across its major features:

### 4.3.1 Machine Learning Optimization

- The disease detection model is built on MobileNetV2 architecture, trained using TensorFlow and Keras. The model achieved an accuracy exceeding 92% in detecting lumpy skin disease based on annotated image datasets.
- To enable fast and efficient inference directly on mobile devices, the trained model was converted to the TensorFlow Lite format. Quantization and model compression techniques were used to reduce model size and latency, making it suitable for low-power environments without sacrificing predictive accuracy.
- Image preprocessing such as resizing, normalization, and format conversion is handled on the client side, reducing backend load and improving response time.

### 4.3.2 Security and Authentication

- All sensitive API endpoints, including those for diagnosis results and appointment scheduling, are protected using JWT-based authentication.
- User credentials and tokens are encrypted both in transit (via HTTPS) and at rest, using standard cryptographic protocols.
- Role-based access is enforced across the backend using guards and interceptors in NestJS to prevent unauthorized access.

### 4.3.3 On-Device Image Processing

- Image capture and transformation tasks are performed locally on the user's device. This approach ensures that raw image data is not unnecessarily transmitted, improving privacy and reducing data usage.
- Processed images are passed to the model in real time, which enables diagnosis even with intermittent connectivity.

### 4.3.4 Geolocation and Visualization

- The app captures user-approved GPS data when a disease report is filed. This data is used to dynamically plot outbreak clusters.

- Using Mapbox and heatmap rendering libraries, cases are visualized with geospatial accuracy, allowing users and authorities to track disease spread in real time.

### 4.3.5 Appointment Booking and Notifications

- A calendar-based interface lets users book veterinary appointments with time-slot availability pulled from the backend.
- Real-time push notifications inform users of appointment confirmation, updates, or cancellations. These are handled via Firebase Cloud Messaging (FCM).
- Backend scheduling APIs include logic for conflict detection and auto-updating the status of appointment slots.

## 4.4 Sample Screenshots

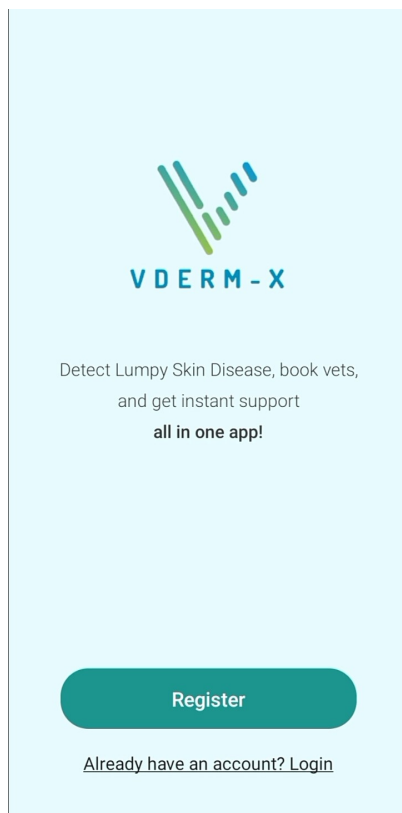


FIGURE 4.1: Launch Screen of the VDERM-X

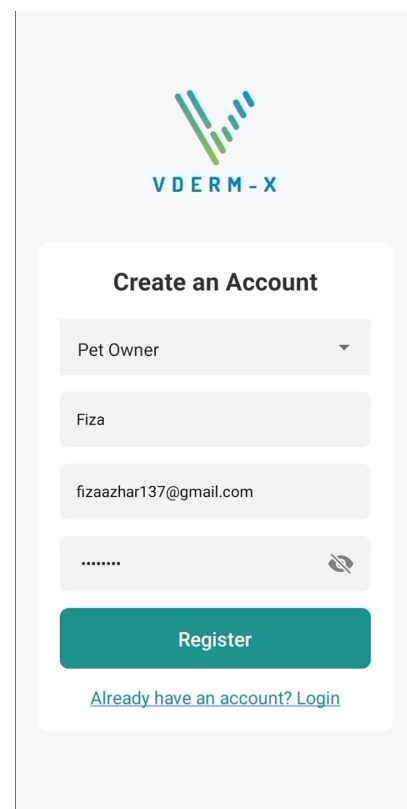


FIGURE 4.2: Registration Screen of the VDERM-X

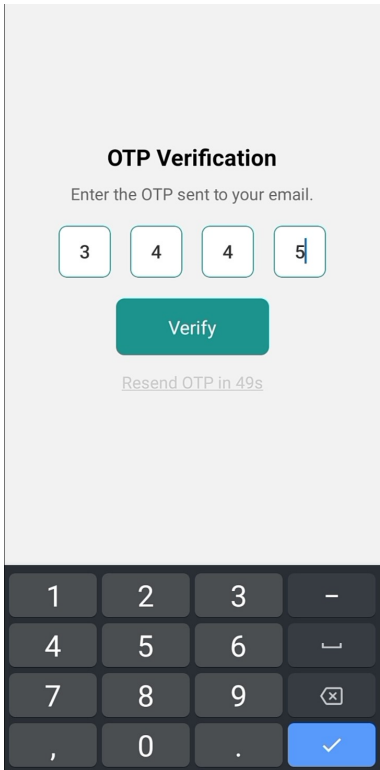


FIGURE 4.3: OTP Verification Screen of the VDERM-X

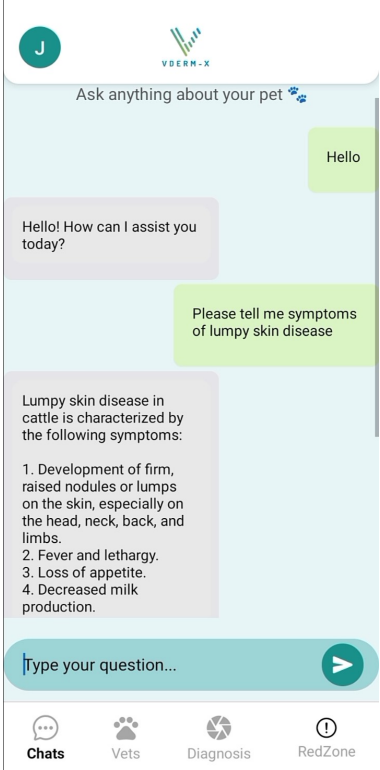


FIGURE 4.4: Chatbot Screen of the VDERM-X

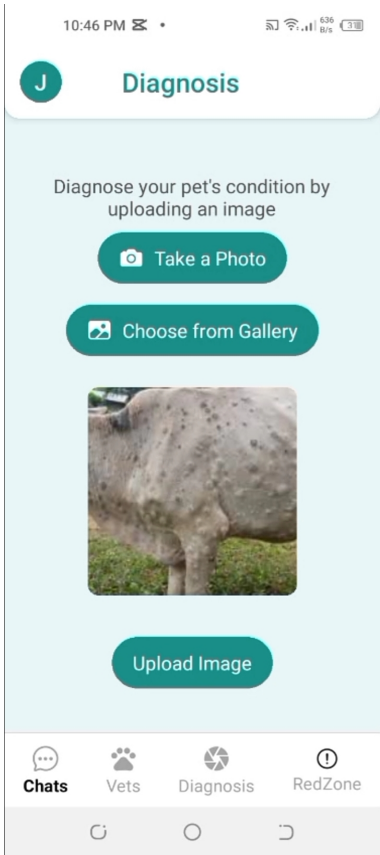


FIGURE 4.5: Image Detection Screen of the VDERM-X



FIGURE 4.6: Disease Outbreak Screen of the VDERM-X

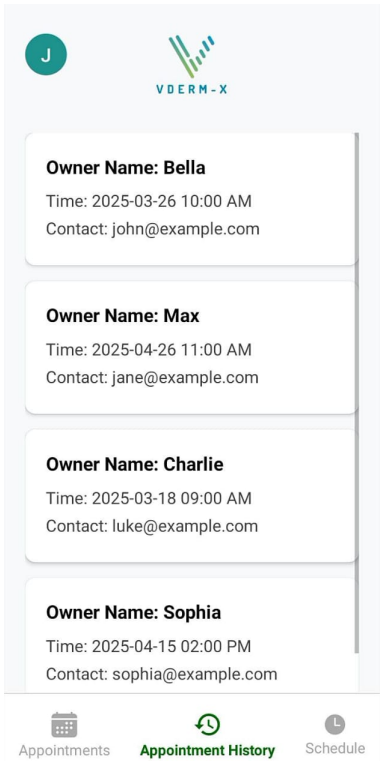


FIGURE 4.7: Vet Appointment History

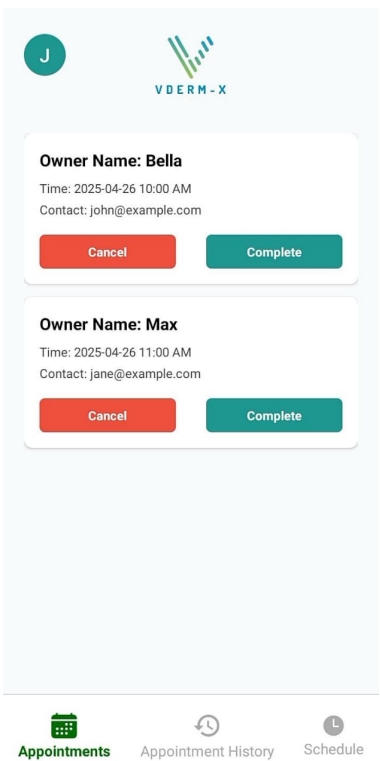


FIGURE 4.8: Vet Scheduled Appointments

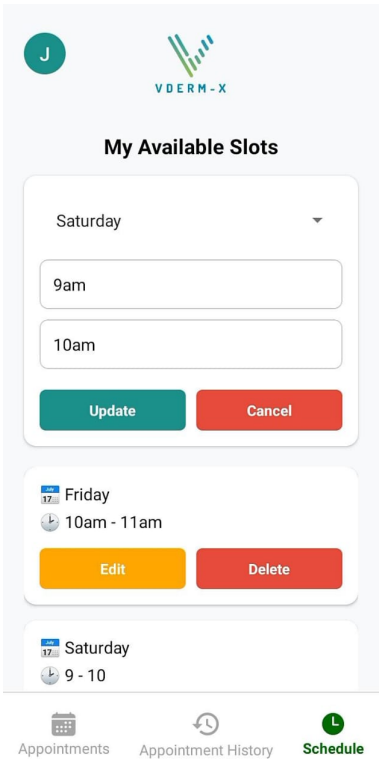


FIGURE 4.9: Vet Slots Addition

# Chapter 5

## Testing

### 5.1 Testing Strategy

To ensure the quality and performance of the application, we implemented a multi-level testing approach:

- **Unit Testing:** Focused on verifying the smallest components like user registration, OTP verification, and disease detection logic.
- **Integration Testing:** Ensured the flow between multiple components—such as registration to OTP to dashboard—is seamless.
- **System Testing:** Validated the application against its functional and non-functional requirements.
- **User Acceptance Testing (UAT):** Conducted with real users including testers and domain experts to validate end-to-end usage.

#### Example Scenario: Vet Approval Workflow

- **Unit Test:** Tested vet registration form validation (fields like email, image, contact).
- **Integration Test:** Verified that successful signup triggered an email to the admin.
- **System Test:** Simulated complete flow from vet signup → admin approval → vet login and schedule update.
- **UAT:** Had 2 participants (acting as admin and vet) go through the workflow and report timing issues, errors, or UX suggestions.

## 5.2 Test Cases and Results

| Test ID | Description                               | Steps                                      | Expected Result              | Actual Result           |
|---------|-------------------------------------------|--------------------------------------------|------------------------------|-------------------------|
| TC001   | Verify login with valid credentials       | Enter valid email and password             | User logged in               | Login successful        |
| TC002   | Validate image upload with correct format | Upload supported image file                | Image uploaded               | Successfully uploaded   |
| TC003   | Disease detection from image              | Upload image for analysis                  | Disease is detected          | Disease detected        |
| TC004   | Booking vet appointment                   | Select vet slot and book                   | Appointment created          | Appointment booked      |
| TC005   | User registration with OTP                | Register and verify OTP                    | User registered              | Registration successful |
| TC006   | Chatbot response check                    | Ask a health-related question              | Bot responds correctly       | Query answered          |
| TC007   | Appointment scheduling by vet             | Vet chooses and updates schedule           | Schedule updated             | Schedule updated        |
| TC008   | Admin vet approval flow                   | Vet signs up → admin gets email → approves | Vet is approved and notified | Vet approved            |
| TC009   | Vet logs in after approval                | Approved vet logs in                       | Login and access features    | Success                 |

|       |                                 |                                                 |                            |                                  |
|-------|---------------------------------|-------------------------------------------------|----------------------------|----------------------------------|
| TC010 | Vet tries login before approval | Vet signs up but tries to login before approval | Access denied              | Error shown                      |
| TC011 | Upload unsupported file format  | Upload '.exe' or unsupported format             | Rejected with error        | Error displayed                  |
| TC012 | Password reset process          | Enter email → get reset link → reset password   | Reset successful           | Password reset done              |
| TC013 | Invalid OTP validation          | Enter wrong OTP during registration             | Show invalid OTP message   | Error message shown              |
| TC014 | Admin rejects vet request       | Admin clicks reject on vet email                | Vet is not added to system | Rejected with email notification |

TABLE 5.1: Comprehensive Test Cases

## 5.3 Bug Fixes / Issue Tracker

| Bug ID | Module                  | Issue Description                        | Status                             |
|--------|-------------------------|------------------------------------------|------------------------------------|
| BUG001 | Login                   | Case-sensitive email failed login        | Resolved                           |
| BUG002 | Upload                  | ‘.jpeg’ files failed due to MIME issue   | Fixed                              |
| BUG003 | OTP                     | OTP expiry too fast (15 sec)             | Increased to 60 sec                |
| BUG004 | Chatbot                 | Delay in response under slow networks    | Added loading spinner              |
| BUG005 | Vet Signup              | Certificate file not accepted (PDF only) | Allowed multiple file types        |
| BUG006 | Admin Email             | Admin not receiving vet approval emails  | Fixed email sending logic          |
| BUG007 | Vet Access Pre-approval | Vets accessing dashboard before approval | Restriction added with error toast |
| BUG008 | Appointment Double Book | Same slot booked by multiple users       | Slot locking logic added           |

## 5.4 Tools Used

### Postman

Postman was used extensively to test backend API endpoints. This includes testing for:

- User and vet registration APIs
- OTP verification
- Admin vet approval
- Image upload and response handling
- Disease detection
- Chatbot request/response testing

It allowed for quick debugging, header testing, response visualization, and checking status codes and error handling logic.

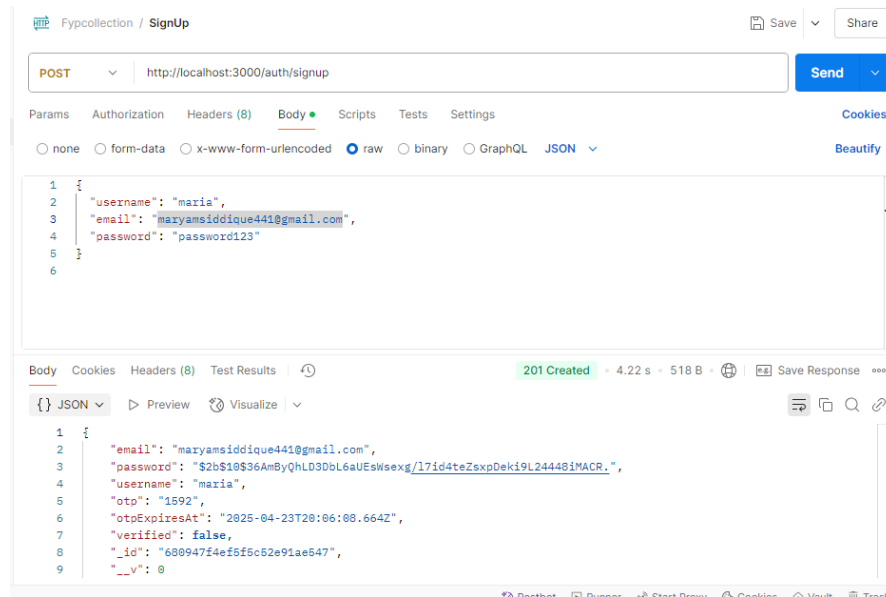


FIGURE 5.1: Signup API Tested Using Postman

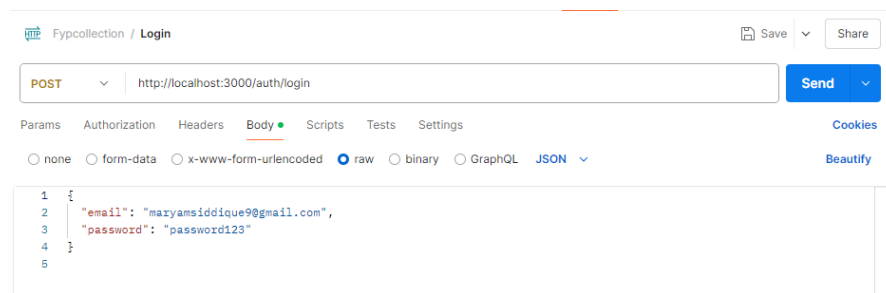


FIGURE 5.2: Login API Tested Using Postman

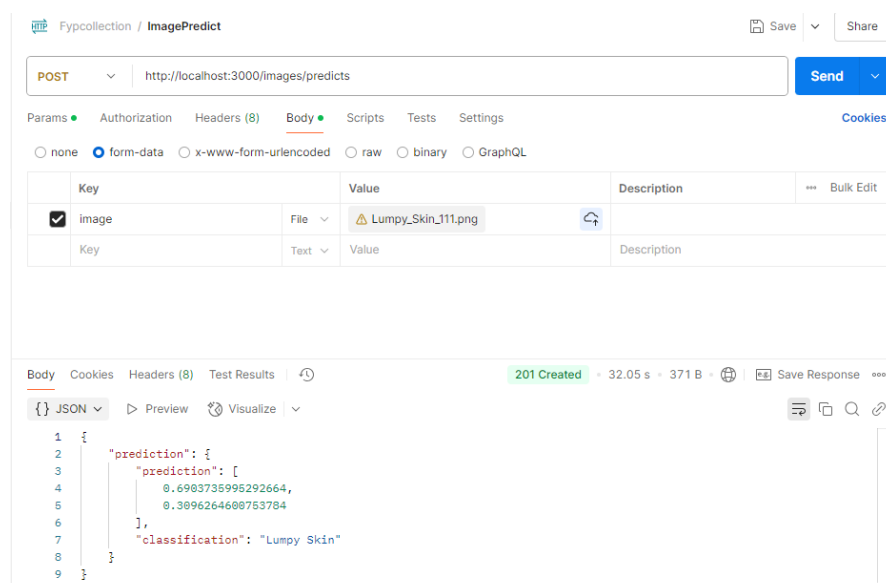


FIGURE 5.3: Disease Detection API Tested Using Postman

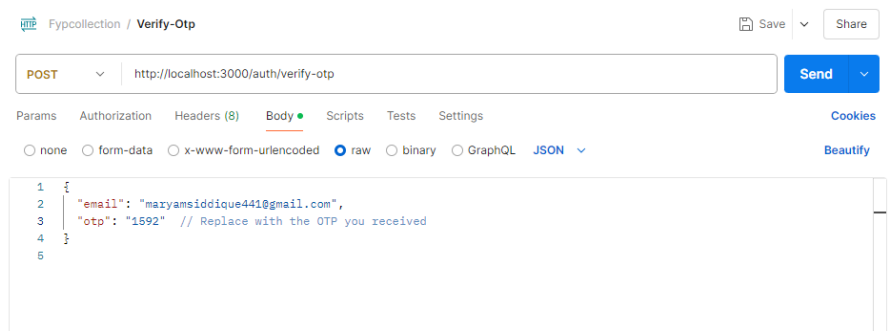


FIGURE 5.4: Verify OTP API Tested Using Postman

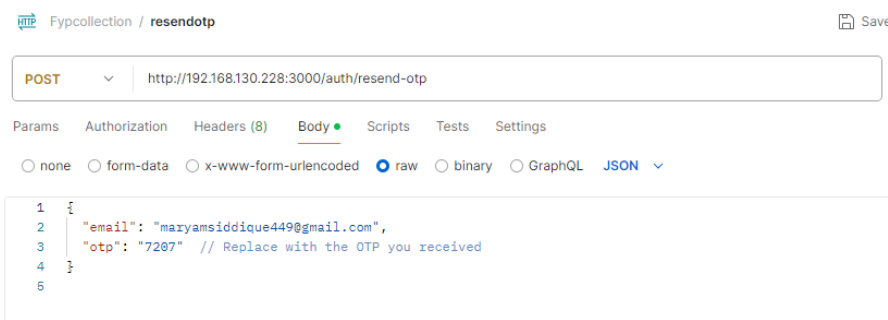


FIGURE 5.5: Resend Otp

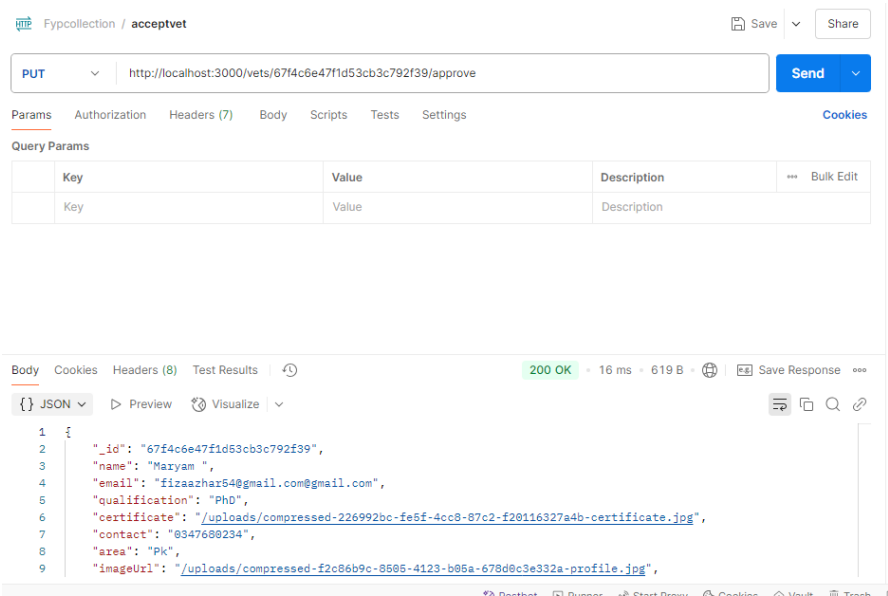


FIGURE 5.6: Vet Approval API

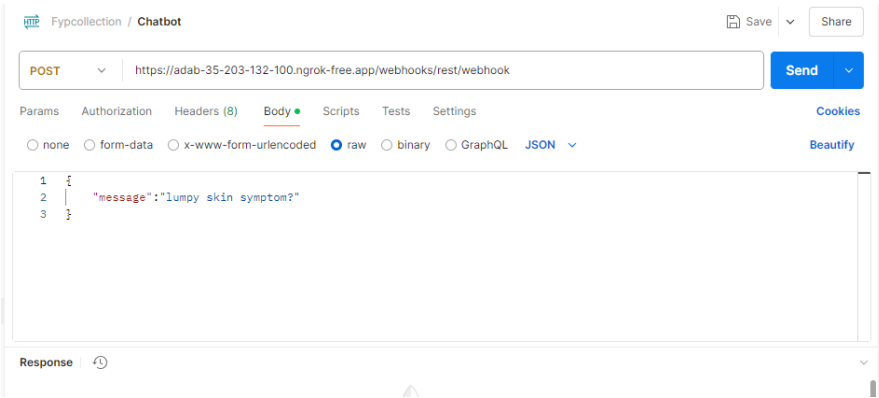


FIGURE 5.7: Chatbott API

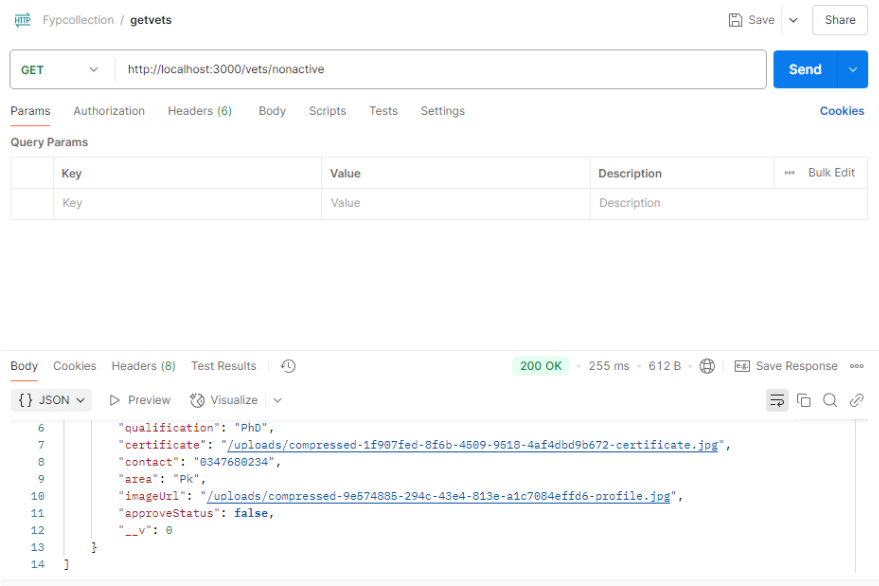


FIGURE 5.8: Get all vets API

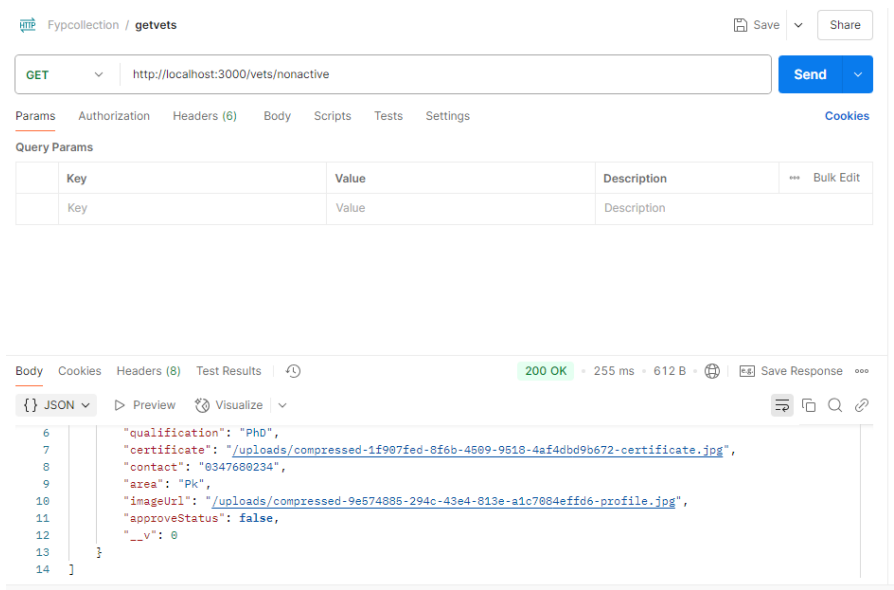


FIGURE 5.9: Get Appointment of specific vet

# Chapter 6

## Deployment

### 6.1 Deployment Steps

The deployment process of the VDerm-X system consists of several key steps that involve setting up the hosting environment, deploying the backend and frontend components, and configuring all necessary dependencies. Below is a detailed guide to deploying the VDerm-X system:

#### 6.1.1 Hosting Platforms

##### **Frontend Deployment:**

The mobile frontend of the VDerm-X application is a React Native-based project, which can be deployed on both Android and iOS platforms. The following platforms are typically used for hosting and distributing the app:

- **Google Play Store:** The Android version of the app is deployed to the Google Play Store, which provides an easy way for Android users to download and update the application. Developers need to generate a signed APK or AAB file, configure the store listing, and upload the app.
- **Apple App Store:** The iOS version is deployed through the Apple App Store. Developers have to adhere to Apple's strict app submission requirements and testing, signing, and uploading requirements for the app.
- **Expo (optional for development):** During development, Expo can be utilized for quicker deployment, testing, and distribution without the need for a complete native build. It is ideal for rapid testing and iteration but not for production releases.

**Backend Deployment:**

The VDerm-X system's backend API is hosted on a cloud platform like Amazon Web Services (AWS), Google Cloud Platform (GCP), or Microsoft Azure. Such platforms provide scalable hosting options for hosting Node.js-based backend applications.

- **Amazon Web Services (AWS):** AWS is widely used as a backend host. For Node.js apps, AWS Elastic Beanstalk is used for simple deployment and Node.js service management. AWS EC2 instances can also be employed for more tailored configurations.
- **Google Cloud Platform (GCP):** GCP's App Engine or Compute Engine can be utilized to host Node.js applications, offering managed services and scalability.
- **Microsoft Azure:** Microsoft Azure offers App Services for deploying Node.js applications, with a variety of scalability options and integrated services.

**Database Deployment:**

MongoDB, being the NoSQL database for use in the VDerm-X system, is deployable to the using cloud-based platforms:

- **MongoDB Atlas:** MongoDB Atlas is a cloud database service that is fully managed and takes care of database scaling, security, backups, and updates.
- **Self-Hosted MongoDB (Optional):** MongoDB can also be hosted on a virtual machine using AWS EC2, GCP, or Azure. However, this requires manual setup and maintenance of the database instance.

**Machine Learning Model Deployment:**

For efficient on-device machine learning model inference, the MobileNetV2 model is converted to TensorFlow Lite for deployment in the mobile app. However, any updates to the model, such as retraining or fine-tuning, require redeploying the updated TensorFlow Lite model.

For server-side inference, the model can be deployed on a cloud GPU-enabled instance (e.g., AWS EC2 with GPU, GCP AI Platform) to handle any heavy-lifting inference tasks that cannot be processed on-device.

### 6.1.2 Dependencies

The successful deployment of the VDerm-X system requires several dependencies, including both runtime libraries and deployment tools. Below is a list of critical dependencies for both the frontend and backend components:

- **Frontend Dependencies:**

- React Native (with TypeScript)
- Expo CLI (for development)
- React Navigation (for navigation)
- Redux (for state management)
- TensorFlow Lite (for mobile inference)
- Mapbox SDK (for disease mapping)
- Firebase SDK (for push notifications)

- **Backend Dependencies:**

- Node.js (LTS version)
- Express.js (web framework)
- NestJS (for backend structure)
- MongoDB Node.js driver (for database interactions)
- JWT (for authentication)
- Axios (for external API calls)

- **Database Dependencies:**

- MongoDB Atlas (for cloud database management)
- Mongoose (for interacting with MongoDB in Node.js)

- **Machine Learning Dependencies:**

- TensorFlow and Keras (for model training)
- TensorFlow Lite (for on-device model inference)
- Google Colab (for cloud-based training)

- **Version Control:**

- Git (for version control)
- GitHub (for collaboration and repository management)

## 6.2 User Instructions

The VDerm-X system is intuitive and easy to use, allowing simple access to all features, from disease detection to booking veterinary appointments. The following is a step-by-step guide for users on how to log in, move around the app, and utilize its features.

### 6.2.1 Login and Account Creation

When launching the VDerm-X app, users will be greeted with the login screen. The following are the login or sign-up steps:

1. Launch the VDerm-X app on your device.
2. On the welcome page, tap on the Login option to input your login details (email and password). Tap on the Sign Up option to sign up for a new account instead.
3. In case of signing up, input your required details like your email, password, and optional user profile information.
4. A verification code will be sent to your email. Enter the verification code in the provided field.
5. Once logged in, you will be directed to the home screen of the app, where all the primary features are accessible.

### 6.2.2 Navigating the App

The VDerm-X app is designed with a simple and intuitive navigation structure. The primary navigation components are as follows:

- **Home Screen/Chatbot Consultation:** From the home screen, users can navigate to the chatbot consultation module by tapping on the chatbot icon. This feature provides a 24/7 assistant capable of answering veterinary-related queries. Users can inquire about symptoms, treatment options, and general animal care. The chatbot utilizes AI to provide accurate responses, and it is especially useful when immediate veterinary help is needed. It also offers bilingual support where users can also query in Urdu language.
- **Disease Detection:** Another navigation choice on the home screen takes one to the disease detection module. The users are able to load an image of

the animal skin or capture a real-time photograph through the device's camera. The system undergoes processing on image using its machine learning model to identify the presence of lumpy skin disease, offering an interpretation of the findings as well as recommendations for further action.

- **Appointment Booking:** Users can also go to the From the home screen the appointment booking section. This module permits users to navigate a list of local vets by location, choose a time slot, and schedule an appointment for a treatment or consultation.
- **Map for Disease Outbreaks:** The home screen also has access to the map function, which graphically illustrates the dissemination of lumpy skin disease outbreaks. Affected areas can be zoomed in to view heatmaps and observe recent disease outbreak reports in their area. It is helpful for monitoring disease trends and making informed decisions regarding animal care and preventive actions.

### 6.2.3 Using the Disease Detection Feature

To use the disease detection feature:

1. Navigate to the disease detection module.
2. Upload a photo of the animal's skin or use the camera to capture the image.
3. The model will process the image and return a negative or positive response indicating whether lumpy skin disease is present.
4. Recommendations for treatment or further steps will be provided by using chatbot or connecting with vets.

### 6.2.4 Booking Veterinary Appointments

To book an appointment:

1. Navigate to the appointment booking section.
2. Browse through a list of available vets in your area.
3. Select a preferred time slot and confirm the appointment.
4. You will receive an email reminder of your scheduled appointment.

## 6.3 Maintenance Guidelines

Keeping the VDerm-X system guarantees optimal performance, security, and scalability in the long term. Following are guidelines for the maintenance of the application, both on the frontend and backend, along with best practices for data management and model retraining.

### 6.3.1 Frontend Maintenance

- **App Updates:** Give users updates periodically, dealing with bug fixes, new functionality, and optimization improvements. Test the updates on diverse devices for assurance of compatibility and performance.
- **Compatibility Testing:** With every release of the newer versions of Android and iOS, test the application to be sure it will run on the current operating systems. This encompasses scanning for older API usage and configuring the app according to updated platform guidelines.
- **User Alerts and Reminders:** Install a system of timely user reminders and alerts pertaining to appointments, future treatments, and disease-related information. This facility keeps users apprised of significant actions they must perform, like confirming an appointment or following up on a diagnosis.

### 6.3.2 Backend Maintenance

- **Server Monitoring:** Monitor the backend server continuously for performance, downtime, and resource usage. Use services like AWS CloudWatch, Google Stackdriver, or New Relic for real-time monitoring.
- **Database Backups:** Back up the MongoDB database regularly to avoid data loss. Configure automatic daily backups and save them in a secure cloud location.
- **API Endpoint Monitoring:** Make sure all API routes are working properly and process error responses in a graceful manner. Add logging for debugging and check for any errors or performance issues.

### 6.3.3 Machine Learning Model Maintenance

- **Model Retraining:** As more data becomes available or the disease detection model's performance degrades, periodically retrain the model with new data. Store versioned models to maintain history and rollback ability.

- **Model Update Deployment:** When deploying a new version of the model, ensure that the new TensorFlow Lite model is correctly integrated into the mobile app. For server-side models, update the model in the cloud environment and test inference accuracy.
- **Model Monitoring:** Regularly monitor the performance of the deployed models to ensure they are performing at optimal levels. Use metrics like prediction accuracy and response time.

### 6.3.4 Security and Data Protection

- **Data Encryption:** Sensitive information like user credentials and medical records must be encrypted in transit (SSL/TLS) and at rest (AES encryption).
- **Authentication and Authorization:** Periodically inspect the JWT authentication mechanisms to make sure they are secure. Token expiration and revocation processes should be implemented to prevent unauthorized access.
- **Compliance:** Periodically inspect the system for compliance with applicable data protection laws like GDPR, HIPAA (if necessary), and local privacy legislation.

# Chapter 7

## Results, Evaluation & Conclusion

### 7.1 Summary of Achievements

The *VDerm-X* application has been able to achieve its initial goals, providing an exhaustive and easy-to-use solution for improving the managements and preventions of lumpy skin disease (LSD) in livestock. Through the incorporation of state-of-the-art machines learning algorithms, mobile app development strategies, and user-oriented features specifically designed for use in the field, the application provides a convenient method for livestock owners to diagnose and prevent the spread of this disease. During the development process, special care was taken to address the particular difficulties of farmers and veterinarians so that the end result is both technically sound and sensible for daily use.

The *VDerm-X* project is founded on the principle of giving livestock owners affordable tools to track and control the health of their herds. Through the use of AI-driven image analysis, a smart veterinary chatbot, and a simple appointment scheduling system, the app offers a multi-dimensional disease control strategy. The solution also makes use of offline capabilities and geographic tracking to guarantee that it can still be effective even in remote areas where network connectivity is often a problem. The next subsections present the major elements of the *VDerm-X* application and how they help accomplish the overall objectives of the project.

#### 7.1.1 AI-Based Disease Detection

- The detection engine core of the *VDerm-X* software is based on the MobileNetV2 model, which is a cutting-edge convolutional neural network (CNN) architecture that is very efficient and accurate. The model is trained specifically on a bespoke dataset of images showing different stages of lumpy

skin disease in cattle, which makes it very specialized for this specific task. Through harnessing the power of TensorFlow Lite, the app provides instantaneous predictions, making sure that users can rapidly diagnose the disease from the comfort of their mobile devices even in low-resource settings. The detection engine works effectively on static images as well as streaming camera feeds, enabling users to take photos of their animals or stream live video for instant analysis. Before making forecasts, the application uses a chain of pre-processing filters and normalization methods directly on the device to improve the accuracy of the model and make sure that the output is as valid as can be. This on-device computing ensures to reduce the burden on the backend infrastructure, resulting in quicker response times and better user experience. Moreover, the system will be able to adjust to changing environmental conditions and different lighting conditions. The model has been tuned to support a broad spectrum of image qualities, from high-resolution images to nighttime lighting, so it can deliver strong performance in the real world. This adaptability is particularly essential where cameras may be of varying quality in rural settings but the demand for precise detection of disease remains high.

- With an impressive classification accuracy exceeding 92%, the AI-powered detection system is a potent weapon for livestock farmers to detect likely instances of LSD early on, and thereby enable prompt intervention and treatment. The accuracy of the app, coupled with its rapidity and simplicity of use, makes it an invaluable tool for averting the spread of the disease and minimizing the economic burden of infections in livestock.

### 7.1.2 Interactive Veterinary Chatbot

- The *VDerm-X* app features an AI-powered veterinary chatbot that delivers instant, actionable advice to end-users. Geared to resolve frequent queries associated with lumpy skin disease as well as animal health in general, the chatbot leverages a knowledge base honed from a set of more than 1,000 QA experiences with veterinarians. The outcome is an accurate and accessible conversational interface through which users can engage with the system naturally without requiring a veterinarian to answer every question.
- This chatbot is not just for basic questions but also seamlessly integrates with the disease detection results, providing contextual advice based on the specific output. For instance, if the detection engine finds possible indications

of LSD in an animal meant for livestock, the chatbot is able to provide corresponding treatment options, recommend prevention, and even walk users through what's next, including calling a vet or quarantining the infected animal. This characteristic greatly minimizes the reliance on face-to-face consultations, enabling users to act promptly.

- The chatbot continues to improve with machine learning methods, using user input and interaction data to make its responses better over time. Through ongoing learning from actual conversations, the chatbot becomes more effective at delivering personalized advice, making it a useful tool for livestock owners at any point in disease management.
- The user-friendliness, along with the intelligence of the chatbot, makes it a standout feature of the app. Through the delivery of real-time, personalized veterinary advice directly within the app, the chatbot closes the knowledge gap for many cattle owners, especially those in underserved or rural areas where access to professional veterinary care might be limited.

### 7.1.3 Seamless Appointment Booking System

- Another prominent feature of the *VDerm-X* app is its in-built appointment scheduling system, where users can book consultations with qualified veterinarians within the app. This aspect greatly simplifies the process of accessing professional veterinary services, eliminating the need for phone calls or physical visits to veterinary centers to make an appointment.
- The appointment system is user-friendly and simple to use. Users are able to browse a large number of verified veterinary profiles, each listing information regarding the veterinarian's credentials, specialty, and availability. The real-time calendar interface of the app ensures that users can easily find a free time slot that suits their schedule, making it simpler than ever before to schedule a consultation when necessary.
- Additionally, the system sends automatic reminders and notifications to users via Firebase Cloud Messaging to ensure that users do not forget their scheduled appointments. These can be personalized such that users get reminders at their preferred times in advance of the appointment, hence enhancing general user engagement and lessening missed appointment occurrences.

- For maintaining smooth functionality and prevention of scheduling conflicts, real-time updates of availability are managed by the backend of the booking system, checking for conflicting appointments and making required adjustments. This logic also makes sure that appointments get confirmed before they are scheduled on the user's calendar, preventing chances of double-booking or scheduling conflicts.
- Privacy and security are of prime concern in the booking system. All appointment details and personal data are encrypted via industry-standard encryption mechanisms, such that users' sensitive information cannot be accessed without authorization. Data protection and respect for privacy are top priorities during the design of the app, reflecting the increased demand for safe digital solutions for the veterinary field.

#### 7.1.4 Outbreak Mapping and Surveillance

- The *VDerm-X* application has an effective outbreak mapping and surveillance feature that lets the users to submit suspected or confirmed cases of lumpy skin disease within their area i.e longitude and latitude coordinates. The feature is essential for monitoring the geographic distribution of the disease and alerting local authorities to possible outbreaks, facilitating timely response and containment.
- Citizens can provide detailed reports with information on the infected animals specially cattle, as well as GPS-tagged coordinates, and the system is then able to create a reliable real-time map of LSD incidents. This information is displayed as interactive heatmaps, giving a straightforward graphical overview of the spread of disease in terms of time and space. The maps updates automatically as new reports become available, giving a dynamic representation of the outbreak in the areas near to the user.
- The app's surveillance features are user-friendly, making it easy for users to provide inputs towards diseases such as LSD tracking with minimal effort. By allowing users to be involved in disease tracking, the app creates an ambiance of community engagement and shared responsibility, with a potential to improve the efficacy of disease control efforts.
- The outbreak mapping capability is worth gold to individual animal owners as well as to government officials. Local farmers can identify possible outbreaks within their community and act swiftly upon them and can take

precautionary measures, while authorities can utilize the data to initiate specific interventions, e.g., quarantines or vaccination programs. By providing geospatial real-time data, the app enhances situational awareness and decision-making during an outbreak.

### 7.1.5 Robust Offline Functionality

- A key feature of the *VDerm-X* application is its robust offline functionality, ensuring that users in rural and remote areas can still access essential features of the app even when they have limited or no internet connectivity. This functionality is crucial, as many livestock owners operate in regions where reliable network access is unavailable or intermittent.
- Major features of the app, such as access to previous diagnostic results, interaction with the chatbot, and the ability to create new reports, are all fully functional offline. This means that users can continue using the app without disruption, even when they are in areas with no internet coverage.
- To facilitate offline use, all critical data and content are securely cached on the user's device. When the app detects a return to network connectivity, it automatically synchronizes the cached data with the server, ensuring that the user's activities are always up to date. This seamless synchronization process ensures that users never have to worry about losing their progress or missing important updates.
- The offline capabilities of the app are particularly beneficial for farmers and veterinarians working in remote, rural environments where network infrastructure may be lacking. By providing uninterrupted access to key tools, the app ensures that users are always equipped with the resources they need, regardless of their location. This functionality further strengthens the app's value as a practical solution for managing lumpy skin disease in diverse environments.

## 7.2 Challenges Faced

Although the *VDerm-X* project successfully achieved its primary objectives, the development process was not without its challenges. Several obstacles were encountered that required careful consideration and creative solutions. These challenges ranged from technical difficulties in model training and optimization to

user interface design and the integration of various features. The following subsections outline the key challenges faced during the project development and the approaches employed to address them.

### **7.2.1 Data Availability and Quality**

One of the major hurdles in the development of the disease detection model was the limited availability of high-quality labeled data for lumpy skin disease. The model's performance heavily relied on the quality and diversity of the training dataset. However, obtaining a sufficient quantity of images that were properly labeled proved to be challenging, especially given the relatively rare occurrence of the disease in certain regions. The initial dataset, which was small and lacked variability in terms of disease stages and cattle breeds, affected the accuracy of the model's early predictions.

To overcome this issue, data augmentation techniques were employed to artificially increase the size and diversity of the dataset. These techniques involved rotating, flipping, and applying noise to the available images, which helped create a more robust dataset for training. However, despite these efforts, obtaining larger and more diverse datasets remains a significant challenge for the continued improvement of the model. Expanding the dataset with more varied images representing different environmental conditions, lighting, and cattle characteristics is critical for enhancing the model's robustness and ensuring it performs well across various real-world scenarios.

In addition, collaboration with veterinary organizations and farmers is being considered to gather more labeled images and further improve the dataset. These partnerships could provide access to a broader range of high-quality data, which is essential for refining the accuracy of the disease detection system and ensuring it can handle a variety of cases effectively.

### **7.2.2 Model Optimization for Mobile Devices**

While the deep learning model achieved impressive accuracy during training, optimizing it for mobile deployment presented another significant challenge. Mobile devices have limited computational power, memory, and storage compared to servers, which posed a challenge in deploying the model without compromising its performance. The initial version of the model, though accurate, was too large and computationally expensive for efficient mobile use.

The primary approach to solving this problem was converting the model into the TensorFlow Lite format, which is specifically designed for mobile and embedded

devices. This conversion helped reduce the model's memory usage and processing time, making it more suitable for mobile platforms. However, this optimization process also required careful balancing between reducing the model size and maintaining prediction accuracy. Achieving this balance was not straightforward, and multiple iterations of model pruning and quantization were necessary to find the optimal configuration.

Despite these efforts, further improvements can still be made, particularly in terms of optimizing the model for even lower-end mobile devices with more constrained resources. Techniques such as knowledge distillation, where a smaller model learns from a larger, more complex model, are being explored as potential solutions for improving performance on a wider range of devices without sacrificing accuracy.

Additionally, ensuring that the optimized model can perform disease detection in real-time on mobile devices, with minimal latency and efficient battery usage, remains an ongoing challenge that will require continued optimization as mobile hardware and software evolve.

### **7.2.3 Integration of Chatbot**

The integration of the AI-powered veterinary chatbot into the *VDerm-X* app was another area where technical challenges were encountered. The chatbot was designed to provide accurate and conversational guidance based on veterinary knowledge, including advice on managing lumpy skin disease and related treatments. One of the key difficulties in building the chatbot was fine-tuning the language model to provide domain-specific responses that were both accurate and contextually relevant.

Training the chatbot on a comprehensive dataset of veterinary-related questions and answers was a lengthy and resource-intensive process. A substantial amount of domain-specific data was required to ensure that the chatbot could handle a wide variety of queries, including those related to diagnosis, treatment options, and preventive measures. The limited availability of high-quality veterinary data for training the language model made this process more difficult, and a significant amount of manual curation and filtering of the training data was necessary to ensure the chatbot's responses were reliable.

Moreover, ensuring the smooth integration of the chatbot with the mobile app posed additional challenges. The chatbot had to be connected to the app's backend via REST APIs, and maintaining consistent performance across different mobile platforms (iOS and Android) required careful testing and optimization. Furthermore, after gathering user feedback, we recognized the need to implement the

chatbot in Urdu, in addition to English. This added complexity, as the model had to be retrained and adjusted for a new language, which required additional training data and language-specific considerations.

The challenges in chatbot integration also extended to performance optimization, as ensuring that the system could handle a large number of user interactions in real-time without significant delays was a critical requirement. We had to strike a balance between providing detailed, accurate responses and maintaining a fast, responsive user experience. This was achieved through careful management of server resources and optimization of the API calls to minimize latency.

#### 7.2.4 User Interface Design

Designing a user-friendly, intuitive, and responsive interface for the *VDerm-X* app proved to be a challenging task. Given that the app was designed to serve a diverse user base, including farmers, veterinarians, and field agents, the interface had to be simple enough for non-technical users to navigate while also providing advanced features for professionals. Ensuring that the app functioned well across various devices and screen sizes, particularly given its multi-platform support for both iOS and Android, added to the complexity of the UI design process.

One of the primary concerns was ensuring that the app would provide a seamless user experience on different devices, ranging from smartphones with smaller screens to tablets with larger displays. The design had to adapt to various screen resolutions, ensuring that all interactive elements were accessible and functional regardless of the device used. This required a significant amount of testing and adjustments to the layout and functionality, with particular attention paid to how key features, such as disease detection and appointment scheduling, were displayed on different screen sizes.

Another challenge was the iterative process of refining the user interface based on feedback from potential users. Initial versions of the app were tested with target users, including farmers and veterinarians, who provided valuable insights into the app's usability. Based on their feedback, several UI elements were redesigned to improve clarity and ease of use. For instance, the appointment booking process was simplified, and the chatbot interface was made more accessible. Additionally, accessibility features, such as larger fonts and high-contrast modes, were incorporated to ensure that the app could be used by individuals with varying levels of vision or motor skills.

Despite these efforts, the need for continuous improvement in UI design remains, as user expectations evolve and new features are added. The design team continues

to focus on optimizing the user experience by implementing updates that address emerging usability issues and accommodate feedback from real-world users.

## 7.3 Lessons Learned

Throughout the development of the *VDerm-X* application, the team encountered numerous challenges, but each of these presented valuable learning opportunities. These lessons ranged from technical insights in machine learning deployment and app development to understanding the critical importance of user interaction. Below are some of the key lessons learned during the project that will guide future iterations and innovations in similar systems.

### 7.3.1 Importance of Data Quality and Diversity

One of the most significant lessons learned during the development of the *VDerm-X* application is the critical role data quality plays in machine learning. The performance of machine learning models is highly dependent on the richness and accuracy of the data on which they are trained. The dataset used for the disease detection model was initially limited in size and diversity, affecting the model's generalization to real-world scenarios. The images available were not sufficiently varied in terms of environmental conditions, lighting, and different stages of lumpy skin disease (LSD).

To address this, data augmentation techniques were employed to artificially expand the dataset. These methods, such as rotating images, applying color shifts, and cropping, helped make the model more robust. However, it became clear that relying on augmentation alone was insufficient for handling edge cases. Moving forward, it will be crucial to invest in obtaining a larger and more diverse set of images, especially from different geographic regions, and featuring a wider range of cattle breeds and stages of disease. This expanded dataset would improve the model's ability to detect LSD in diverse conditions and help ensure that the system performs well globally, accounting for variations in animal health and environmental factors.

Furthermore, another lesson learned is that data collection is an ongoing process. Continuous collaboration with veterinary practitioners and farmers, who are key stakeholders, is essential for regularly updating the dataset. Involving local authorities and farmers in data collection efforts will also enhance the model's diversity, making the system even more adaptable to new challenges.

### 7.3.2 User-Centered Design

A major lesson from the development of *VDerm-X* was the importance of placing the user at the center of the design and development process. Engaging users early and often through user testing and feedback loops is essential for building a product that truly meets their needs. Early user testing revealed several issues with the app's initial design, such as the complexity of the appointment booking system and difficulties in navigating diagnostic results. Without regular user input, these issues might have gone unnoticed until later stages, possibly affecting the app's usability and user satisfaction.

By conducting frequent user interviews and testing, the team was able to identify these pain points and refine the app's features. For example, simplifying the process of booking veterinary appointments and enhancing the chatbot's conversational flow were direct results of this iterative testing and feedback. This focus on user feedback ensured that the app was intuitive and accessible for its target audience, including farmers and veterinary professionals with varying levels of technological expertise.

In addition to improving usability, a user-centered approach also helped refine the design by making the app more visually appealing and easier to navigate across various screen sizes and devices. User feedback led to enhancements in the layout, font sizes, button placements, and overall design, making the app more responsive and user-friendly. Moving forward, incorporating user-centered design principles will be vital in ensuring that future updates to *VDerm-X* remain aligned with the needs of its users.

### 7.3.3 Offline Functionality is Crucial

Another important lesson learned was the significance of offline functionality in applications designed for rural or remote areas with limited or no internet connectivity. Many of the target users for *VDerm-X*, such as farmers and field veterinarians, operate in areas where consistent internet access is not guaranteed. Initially, the app was designed with an assumption that users would always be online, but this proved to be an unrealistic assumption given the target environment.

The ability to use core features of the app offline became an essential requirement, especially for diagnostic capabilities and access to historical data. For instance, farmers in rural areas may need to diagnose their cattle in the absence of a stable network connection. Implementing offline functionality meant that users could continue to access past diagnoses, interact with the chatbot, and even report cases

of LSD without an active internet connection. Once the device is connected to the internet again, all offline data is synchronized to the cloud, ensuring that user actions are not lost.

This lesson reinforced the idea that ensuring access to essential features in environments with limited connectivity is crucial for the success of mobile health applications. As the app evolves, maintaining this offline functionality, and expanding it to include more features, will be a key consideration to ensure accessibility for underserved populations.

### 7.3.4 Scalability Considerations

As *VDerm-X* gained traction and began to expand its user base, it became evident that scalability needed to be considered from the very beginning. An application that integrates machine learning models and processes large amounts of data requires careful planning to ensure it can handle increasing demand. The machine learning model and the backend systems needed to scale efficiently as more users began submitting diagnostic images, booking appointments, and interacting with the chatbot.

Initially, the system was designed to handle a smaller, localized user base, but as the app began to grow, it quickly became clear that scalability would be a challenge. The backend architecture had to be adjusted to handle the increased load without compromising performance. This included optimizing the use of cloud resources, improving database management, and ensuring that the machine learning model could scale without performance degradation. Additionally, the system needed to handle an increase in the number of concurrent users interacting with the chatbot and diagnostic features.

From this experience, it was clear that scalability should be considered during the design phase of any machine learning-based application. It's essential to build the infrastructure in a way that can handle a growing user base, particularly as demand for features like disease detection and veterinary appointments increase. In future projects, scalability considerations will be prioritized to avoid performance bottlenecks and ensure smooth user experiences as the app scales up.

### 7.3.5 Cross-Disciplinary Collaboration

A significant lesson from the development of *VDerm-X* was the importance of fostering cross-disciplinary collaboration between different experts. Building an application that combines machine learning, natural language processing (NLP), and veterinary care required the expertise of various professionals in these domains.

It became clear that close collaboration between data scientists, AI experts, and veterinary professionals was essential to ensure the accuracy and relevance of the system's features.

For instance, working with veterinarians allowed the team to understand the nuances of LSD diagnosis and treatment, which in turn helped tailor the chatbot's responses and the diagnostic model's approach. Additionally, veterinarians provided valuable feedback on the clinical aspects of the disease and helped ensure that the app provided practical, evidence-based advice. Similarly, collaboration with data scientists and machine learning engineers helped optimize the backend model to handle the constraints of mobile devices, allowing for seamless integration of the diagnostic engine within the app.

This experience underscored the importance of establishing strong communication channels and working collaboratively across disciplines. Future iterations of *VDerm-X*, as well as similar projects, will benefit from ongoing collaboration with experts from multiple fields, as this interdisciplinary approach is key to creating effective, impactful solutions.

## **7.4 Future Enhancements**

While *VDermX* has successfully met its initial goals, there is ample opportunity for further development to enhance the application's impact and capabilities. With the aim of enhancing user experience, expanding the range of disease detection, and incorporating enhanced features, some major areas have been outlined for future upgrades. These upgrades will help expand the app's functionality, ensure that it remains at the forefront of technology, and render it more useful to users. Some of the most promising areas for future work include:

### **7.4.1 Improved Machine Learning Models**

Although the existing MobileNetV2 model has demonstrated encouraging performance with an accuracy of more than 92%, there is sample scope for improvement in its performance. A possible area of future research is the inclusion of additional, more varied data, especially real-world data collected from different geographical regions and different cattle breeds. This would provide a more diverse dataset for training, improving the model's ability to generalize to a wider range of conditions.

Furthermore, leveraging more advanced deep learning architectures such as custom convolutional neural networks (CNNs) or even transformer-based models could potentially improve the disease detection accuracy. These architectures are known for their ability to capture intricate patterns in images, which could enhance

the model's sensitivity and specificity in detecting lumpy skin disease (LSD) and other conditions. Additionally, experimenting with techniques like transfer learning—using pre-trained models fine-tuned on specific datasets—could also help in boosting the model's performance, especially when data availability is limited.

Another important aspect for future improvement is model optimization for mobile devices. While the app currently uses TensorFlow Lite to ensure efficient performance on mobile devices, further refinements in model pruning and quantization can reduce model size and computational load while preserving accuracy, ensuring that the app remains responsive and resource-efficient.

### **7.4.2 Expanded Disease Detection Capabilities**

The current focus of *VDermX* is primarily on the detection of lumpy skin disease, but there is significant potential to broaden the range of conditions that the app can detect. Expanding the app's diagnostic capabilities to include other common skin diseases in livestock, such as foot-and-mouth disease or tick-borne diseases, would make the app more versatile and valuable for a wider range of users.

Expanding the disease detection functionality would require additional data collection, training of new models, and continuous retraining to ensure that the new conditions are accurately detected. This could involve collaborating with veterinarians and animal health researchers to gather labeled images of other skin diseases for training purposes. Additionally, integrating features to detect various stages or severity levels of the diseases would add more granularity to the diagnostic process, allowing users to receive more targeted treatment advice.

As part of the expansion, implementing multi-disease detection capabilities would necessitate developing a system to handle multiple models simultaneously, ensuring that the app can intelligently choose the right model based on the symptoms presented by the user. This could also lead to a more sophisticated app architecture that allows for easy addition of new disease models as they become available.

### **7.4.3 Enhanced User Support Features**

One area that could significantly improve the user experience is the enhancement of the app's chatbot and overall support system. While the current chatbot is useful, there is potential for introducing more advanced AI features, such as voice assistants and natural language processing (NLP) improvements. Voice assistants could allow users to interact with the app hands-free, which is especially beneficial for farmers who may be busy working with livestock or those in remote areas without easy access to a keyboard or touchscreen.

Additionally, improving the chatbot's conversational abilities would make interactions smoother and more intuitive. Integrating sentiment analysis could enable the chatbot to better understand and respond to user emotions, providing a more empathetic and personalized experience. A more sophisticated dialogue management system could allow the chatbot to engage in multi-turn conversations, offering more detailed responses based on previous interactions.

Another enhancement would involve incorporating a feedback loop within the chatbot, where users can rate the responses they receive and provide suggestions for improvement. This would enable the chatbot to continuously improve over time by learning from user interactions and incorporating those insights into future responses. The inclusion of a feedback system would also offer a way to fine-tune the chatbot's performance and ensure it remains up-to-date with the latest veterinary practices and user needs.

#### **7.4.4 Integration of Remote Veterinary Services**

A significant future enhancement for *VDermX* is the integration of telemedicine services, especially in regions where access to veterinary professionals is limited or non-existent. By incorporating remote veterinary consultations via video calls or live chat, users would be able to receive professional advice without having to travel long distances to reach a veterinary clinic. This feature would be invaluable to farmers in rural or underserved areas, where veterinary services may be scarce or hard to access.

Video consultations could also enable veterinarians to provide more accurate and real-time diagnostics, potentially facilitating immediate intervention. Furthermore, remote diagnostics could be enhanced by allowing users to upload high-quality images or videos of their livestock, enabling the veterinarian to perform a more thorough evaluation. The integration of such telemedicine features could significantly improve the app's value, making it an indispensable tool for livestock owners facing health challenges in remote locations.

Integrating remote veterinary services would also expand the scope of *VDermX* to cater to a broader range of veterinary needs, beyond just lumpy skin disease. Veterinarians could provide consultations on various diseases, preventive measures, nutrition, and overall livestock health, further enhancing the app's utility.

#### **7.4.5 Further Optimization for Offline Functionality**

While *VDermX* currently offers offline functionality, there are still several areas for improvement that could be made to ensure that users in rural areas with limited

or no internet connectivity can continue to benefit from the app. Future updates could introduce features that allow users to receive real-time alerts or notifications even when they are offline. For example, users could receive emergency alerts related to disease outbreaks or urgent health issues with livestock, even when they do not have a network connection. This feature would significantly enhance the app's usability in remote locations where internet access is intermittent or unreliable.

In addition to notifications, further enhancements to offline functionality could include expanding the synchronization capabilities of the app. For instance, users should be able to book veterinary appointments and track disease outbreaks while offline, with the app automatically syncing the data when the device is connected to the internet. This would ensure that users can continue to use critical features without disruption, regardless of their connectivity situation.

Additionally, expanding offline access to more features, such as diagnostic results and veterinary advice, would further enhance the app's usefulness in areas with unreliable internet access. This could help ensure that the app remains valuable to its users, even in the most underserved regions.

#### **7.4.6 Push Notifications for Timely Alerts**

Push notifications could play a key role in keeping users informed about critical updates. In the future, integrating push notifications would allow the app to send timely alerts to users regarding important issues such as disease outbreaks, available veterinary appointments, reminders for animal care, and any system updates. These notifications could be tailored based on the user's location, livestock health status, or preferences, ensuring that they receive the most relevant information at the right time.

Push notifications could also help improve engagement by reminding users of upcoming veterinary appointments or alerting them to new features and updates within the app. For instance, a farmer who has an upcoming consultation with a vet could receive a reminder notification a day before the appointment. Similarly, if a new disease outbreak is reported in their area, users could be instantly alerted, allowing them to take preventive measures.

Furthermore, integrating push notifications with the app's disease detection capabilities would allow the system to proactively notify users of any potential disease threats or unusual symptoms detected in their livestock, helping them take prompt action. This feature would not only increase user engagement but also enhance

the app's ability to provide real-time, relevant information that can help mitigate potential health risks for livestock.

## 7.5 Conclusion

The VDermX project has effectively developed an innovative mobile app that utilizes cutting-edge technologies, such as machine learning, natural language processing, and offline support, to assist in the early detection and management of lumpy skin disease (LSD) in livestock. Through the use of advanced diagnostic tools, a veterinary chatbot, and a user-friendly appointment scheduling system, the app provides a holistic, practical solution for farmers, veterinarians, and agricultural sector stakeholders. The crux of the app is its disease detection feature, which relies on a MobileNetV2-based machine learning model with a remarkable classification accuracy of over 92% of the capability of machine learning in agriculture applications, particularly in rural areas where access to timely vet care may be restricted. In addition, the inclusion of a chatbot conversational interface, which has been optimized for veterinary consults, adds value to the app by offering users instant, customized advice on disease management, treatment, and prevention techniques. The fact that VDermX can operate offline is also a key accomplishment, as it makes the app available even in remote, poorly connected areas or lack of internet connection. The offline capability is one of the main distinguishing factors, enabling users in remote areas to receive essential information, report disease conditions, and monitor outbreaks of diseases without relying on constant internet connectivity. By offering such functionalities, VDermX can potentially change the way livestock health is being managed, particularly in regions with connectivity issues and poor access to veterinary services.

Yet, the process was not without difficulties. Acquiring data for training the disease detection model was a major challenge, as high-quality labeled data for lumpy skin disease was scarce. Attempts to overcome this challenge, like using data augmentation techniques, were useful but not adequate for optimal model performance. Moreover, fine-tuning the machine learning model for mobile platforms and providing a smooth user interface on different platforms posed technical challenges. In spite of these challenges, the project was able to overcome these issues and deliver its desired results. The experiences gained from these, especially model optimization and user feedback, will be useful insights for future versions of the app. In the future, the VDermX app has tremendous potential for further growth and development. Future updates may involve integrating more disease detection models, increasing the capabilities of the chatbot to enable additional languages

and richer conversational capabilities, and incorporating telemedicine features to enable remote veterinary consultations. In addition, sustained refinement of offline capability and push notification systems will further enhance the app's utility in remote and rural regions, facilitating enhancement of gaps in healthcare delivery. With its continued refinement, VDermX might become a formidable tool in revolutionizing livestock health management and might be used as a model for such applications aimed at other livestock diseases or agricultural issues.

In summary, the VDermX project is a major innovation in the application of technology for enhancing animal health management in the agricultural industry. The combination of machine learning, offline capability, and a veterinary chatbot presents a complete and user-friendly solution to a pressing issue for farmers and veterinarians globally. Although there are opportunities for expansion and improvement, the success of this project illustrates the potential of technology to solve key challenges in livestock health management, especially in rural and underserved communities. With further investment and development, VDermX has the potential to leave a lasting legacy on the agricultural industry, improving both the health and productivity of livestock in areas where they are most needed. The VDermX application represents an important milestone in the convergence of agricultural technology and animal health. Through the application of advanced machine learning algorithms, AI-powered chatbots, and offline capabilities, the project has been able to develop an accessible and efficient tool for early detection and management of lumpy skin disease. This disease, which is a major threat to livestock, especially in developing and rural areas, can now be better monitored and managed through the capabilities offered by VDermX. The success of this project is a testament to the capabilities of technology to transcend conventional obstacles to livestock health care, including restricted veterinary access and absence of viable diagnostic equipment. The project, in spite of the challenges, including limitations in getting adequate high-quality training data as well as having to optimize the mobile app for effective use on resource-poor devices, managed to provide an effective and useful solution. The experience garnered from transcending these hurdles—ranging from technical solutions such as TensorFlow Lite implementation to addressing user interface design feedback—have played a crucial role in shaping the app's development and informing its future upgrades. This process of refinement and improvement through iteration guarantees that the ultimate product is effective and easy to use, serving the needs of its intended audience. As VDermX continues to evolve, some areas for future improvement offer themselves, especially in the extension of its disease detection function, adding new AI-based features, and enhancing the scalability of the app.

Specifically, the extension of the detection system to other significant livestock diseases and incorporating telemedicine functions for real-time consultation will be valuable additions. Further, with increasing user bases, ensuring the scalability of the app to process greater traffic and larger datasets will be critical to its long-term viability. The future holds much promise for VDermX, as it keeps evolving and increasing its functionality to accommodate the changing demands of farmers, veterinarians, and players in the agricultural sector.

In short, the VDermX project has proved that technology can be used to tackle key health problems in cattle husbandry. The successful integration of machine learning, AI, and offline use creates an effective tool that is capable of drastically enhancing the control of lumpy skin disease, especially in areas with poor access to veterinary attention. As the app develops further, it is able to broaden its mandate, give even greater utility to users, and a necessary tool in livestock health management. With continued updates and addition of new functionality, VDermX has the potential to revolutionize the detection, management, and treatment of livestock disease to the advantage of both farmers and the wider agricultural industry for many years to come. The VDermX project is an innovative project that combines cutting-edge technologies with practical agricultural requirements. The app presents a new solution to the long-standing problem of lumpy skin disease in livestock, integrating machine learning for detecting diseases, a veterinary chatbot for professional advice, and offline features that enable the app to be useful even in remote and underdeveloped regions with weak internet connectivity. The creation of this app can potentially make a long-term effect on the agricultural community, equipping farmers and veterinarians with better means to deal with and cure livestock diseases in a more efficient and effective manner. During the duration of the project, various challenges were faced, such as data constraints, model optimization, and user interface design. Nonetheless, through novel solutions like data augmentation and TensorFlow Lite's mobile optimization, these challenges were addressed, and the app was successfully created. This iterative methodology not only resulted in the development of a stable mobile app but also opened up important insights into the app development process that will be used to improve future iterations of the app. The obstacles encountered during the project also served to further highlight the need for collaboration among different fields, such as machine learning, veterinary treatment, and app development, to come up with an integrated solution for cattle disease control. In summary, the VDermX app has succeeded in its initial objective of offering an affordable, AI-powered tool for early detection and control of lumpy skin disease. However, as with all technology, there is always room for improvement. The future upgrades to the app, including

widening its disease detection functions, incorporating telemedicine, and further improving offline capabilities, will only make it more impactful and usable. The success of this project shows the potential for harnessing current technologies to tackle major health issues in agriculture, and VDermX can become a useful tool for farmers, veterinarians, and policymakers as well. Ultimately, VDermX has the potential to transform livestock health management, with an efficient and scalable solution to one of the biggest problems in agricultural health.

## References

1. Y. R. Khan *et al.*, "A review: surveillance of lumpy skin disease (LSD) a growing problem in Asia," *Microb. Pathog.*, vol. 158, p. 105050, 2021.
2. H. T. T. Tran *et al.*, "Lumpy skin disease outbreaks in Vietnam, 2020," *Transbound. Emerg. Dis.*, vol. 68, no. 3, pp. 977–980, 2021.
3. S. Sharma *et al.*, "In depth analysis of recent models for lumpy skin disease virus and stage detection: a comparative study," in *Proc. Int. Conf. Emerging Trends Expert Appl. Security*, 2024, pp. 323–341.
4. P. Hamet and J. Tremblay, "Artificial intelligence in medicine," *Metabolism*, vol. 69, pp. S36–S40, 2017.
5. H. Patel *et al.*, "Role of artificial intelligence in livestock and poultry farming," *Sinkron: Jurnal dan Penelitian Teknik Informatika*, vol. 6, no. 4, pp. 2425–2429, 2022.
6. B. Kayalibay, G. Jensen, and P. van der Smagt, "CNN-based segmentation of medical imaging data," *arXiv preprint arXiv:1701.03056*, 2017.
7. A. Rohan *et al.*, "Application of deep learning for livestock behaviour recognition: a systematic literature review," *Comput. Electron. Agric.*, vol. 224, p. 109115, 2024.
8. S. Fuentes, C. Gonzalez Viejo, E. Tongson, and F. R. Dunshea, "The livestock farming digital transformation: implementation of new and emerging technologies using artificial intelligence," *Anim. Health Res. Rev.*, vol. 23, no. 1, pp. 59–71, 2022.
9. H.-C. Shin *et al.*, "Deep convolutional neural networks for computer-aided detection: CNN architectures, dataset characteristics and transfer learning," *IEEE Trans. Med. Imaging*, vol. 35, no. 5, pp. 1285–1298, 2016.
10. R. B. Appleby and P. S. Basran, "Artificial intelligence in veterinary medicine," *J. Am. Vet. Med. Assoc.*, vol. 260, no. 8, pp. 819–824, 2022.

# Final Year Project Proposal Copy

## Group Members

- Alishba Javed — 2021-CS-205
- Fiza Azhar — 2021-CS-193
- Maryam Siddique — 2021-CS-180
- Muhammad Muaz — 2021-CS-101

## Project Title

VDerm-X

## Description

Lumpy Skin Disease (LSD) is a viral infection that primarily affects cattle but can also pose risks to other animals, including dogs in close proximity. Caused by the *Capripoxvirus*, LSD is characterized by fever, nodules on the skin, mucous membranes, and internal organs, leading to significant economic losses in the livestock industry. Transmission occurs through arthropod vectors, direct contact, and contaminated materials, making LSD a critical concern in veterinary medicine.

Early detection and treatment are essential to prevent its spread and minimize its impact. Given the increasing prevalence of skin diseases among pets, particularly in rural areas with limited veterinary access, there is a strong need for innovative solutions.

This project proposes the development of a mobile application called **VDerm-X**, which leverages advancements in machine learning, deep learning, and natural language processing to provide an efficient and accessible method for detecting and preventing skin diseases in pets, with a special focus on lumpy skin disease in cattle.

The app aims to offer accurate disease identification, verification, and treatment recommendations through a user-friendly platform, helping improve animal health outcomes and supporting veterinary efforts in underserved communities.

## Supervisors

**Supervisor:** Prof. Dr. Muhammad Shoaib

**Co-Supervisor:** Abqa Javed

# Plagiarism Report

The plagiarism report for VDerm-X is shown below.

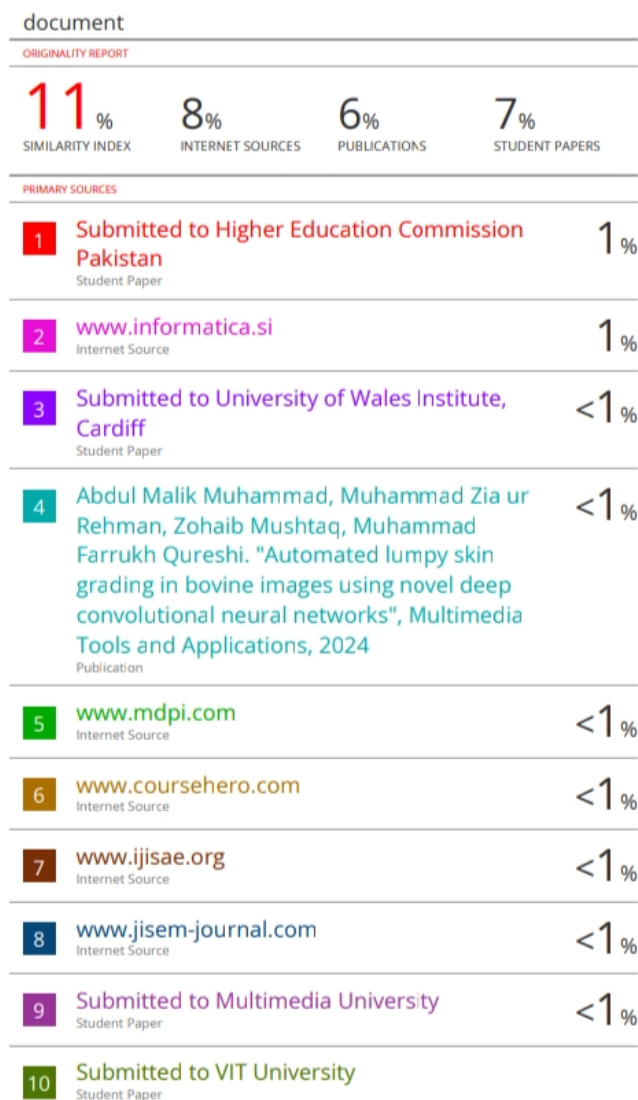


FIGURE 1: Plagiarism Report for VDerm-X

|    |                                                                                                                                                                                                                                                                              |      |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
|    |                                                                                                                                                                                                                                                                              | <1 % |
| 11 | aseestant.ceon.rs<br>Internet Source                                                                                                                                                                                                                                         | <1 % |
| 12 | Longling Geng, Xinzhang Xiong, Zhenyao Liu, Yifan Wei et al. "Evaluation of Smart Home Systems and Novel UV-Oriented Solution for Integration, Resilience, Inclusiveness & Sustainability", 2022 6th International Conference on Universal Village (UV), 2022<br>Publication | <1 % |
| 13 | link.springer.com<br>Internet Source                                                                                                                                                                                                                                         | <1 % |
| 14 | publisher.uthm.edu.my<br>Internet Source                                                                                                                                                                                                                                     | <1 % |
| 15 | fastercapital.com<br>Internet Source                                                                                                                                                                                                                                         | <1 % |
| 16 | Submitted to University of Essex<br>Student Paper                                                                                                                                                                                                                            | <1 % |
| 17 | Submitted to BB9.1 PROD<br>Student Paper                                                                                                                                                                                                                                     | <1 % |
| 18 | Submitted to QA Learning<br>Student Paper                                                                                                                                                                                                                                    | <1 % |
| 19 | resumecat.com<br>Internet Source                                                                                                                                                                                                                                             | <1 % |
| 20 | studentsrepo.um.edu.my<br>Internet Source                                                                                                                                                                                                                                    | <1 % |
| 21 | Amit Gupta, Divya Singh, Richa Gupta, Vikas Tripathi. "Revolutionizing Cattle Health: A Machine Learning Approach to Efficiently Predict Lumpy Disease in Cows", 2023 4th                                                                                                    | <1 % |

FIGURE 2: Plagiarism Report for VDerm-X

# Code Snippets

## Frontend Code Snippets

Below are some important frontend code snippets used in the VDerm-X project.

### User Authentication Component

---

```
const handleSignup = async () => {
  if (!username || !email || !password) {
    Alert.alert("Error", "All fields are required.");
    return;
  }

  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) {
    Alert.alert("Error", "Invalid email address.");
    return;
  }

  const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$/;
  if (!passwordRegex.test(password)) {
    Alert.alert(
      "Invalid Password",
      "Password must be at least 8 characters long and include:\n At least one uppercase lett
    );
    return;
  }

  // Proceed with registration
  try {
    const response = await fetch(`${BASE_URL}/auth/signup`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ username, email, password }),
    });

    const data = await response.json();
    if (response.status === 409) {
      Alert.alert("Error", "Email already exists.");
    } else if (response.ok) {
      Alert.alert("Success", "Verification OTP sent. Check your email.");
      // Navigate to the OTP verification screen
      navigation.navigate("Verify", { email });
    } else {
      Alert.alert("Error", data.message || "Failed to sign up.");
    }
  }
}
```

---

```

    }
  } catch (error) {
    console.error("Signup error:", error);
    Alert.alert("Error", "An error occurred during signup.");
  }
};

```

---

LISTING 1: User Authentication in React Native

## Red Zone Component

---

```

import React, { useState, useEffect } from 'react';
import { View, Text, StyleSheet, Alert } from 'react-native';
import MapView, { Marker } from 'react-native-maps';
import { BASE_URL } from '../config';
import UBottomTabBar from './UBottomTabBar';
import * as Location from 'expo-location';

type RedZone = {
  _id: string;
  latitude: number;
  longitude: number;
  date: string;
};

const AllRedZonesScreen = () => {
  const [redZones, setRedZones] = useState<RedZone[]>([]);
  const [userLocation, setUserLocation] = useState<{ latitude: number; longitude: number } | null>(null);

  // Fetch red zone data
  useEffect(() => {
    const fetchRedZones = async () => {
      try {
        const response = await fetch(`${BASE_URL}/location`);
        if (!response.ok) {
          throw new Error('Failed to fetch red zones');
        }

        const data = await response.json();
        setRedZones(data);
      } catch (error) {
        console.error('Error fetching red zones:', error);
        Alert.alert('Error', 'Could not load red zone data');
      }
    };

    fetchRedZones();
  }, []);

  // Get user location
  useEffect(() => {
    const getLocation = async () => {
      try {
        let { status } = await Location.requestForegroundPermissionsAsync();
        if (status !== 'granted') {
          Alert.alert('Permission Denied', 'Location access is required to show your position');
          return;
        }
      } catch (error) {
        console.error('Error getting location:', error);
      }
    };

    getLocation();
  }, []);

```

```

    }

    const location = await Location.getCurrentPositionAsync({});
    setUserLocation({
      latitude: location.coords.latitude,
      longitude: location.coords.longitude,
    });
  } catch (err) {
    console.error('Location error:', err);
  }
};

getLocation();
}, []);

return (
  <View style={styles.container}>
    <MapView
      style={styles.map}
      initialRegion={{
        latitude: userLocation?.latitude || 20.5937, // Default center (India)
        longitude: userLocation?.longitude || 78.9629,
        latitudeDelta: 0.5,
        longitudeDelta: 0.5,
      }}
    >
      {/* Red Zone Markers */}
      {redZones.map((zone, index) => (
        <Marker
          key={zone._id || index}
          coordinate={{
            latitude: Number(zone.latitude),
            longitude: Number(zone.longitude),
          }}
          pinColor="red"
          title={`Red Zone ${index + 1}`}
          description={`Reported on: ${new Date(zone.date).toLocaleString()}`}
        />
      ))}

      {/* User location marker */}
      {userLocation && (
        <Marker
          coordinate={userLocation}
          pinColor="blue"
          title="You are here"
        />
      )}
    </MapView>

    <UBottomTabBar />
  </View>
);
};
export default AllRedZonesScreen;

```

LISTING 2: Red Zone Component in React Native

## Backend Code Snippets

Below are some important backend code snippets.

### User Authentication Route

---

```
import { Controller, Post, Body, BadRequestException, Put, Param } from '@nestjs/common';
import { UserService } from '../user.service';
import { User } from '../schema/user.schema';

@Controller('auth')
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Post('signup')
  async signup(
    @Body('username') username: string,
    @Body('email') email: string,
    @Body('password') password: string
  ): Promise<User> {
    return this.userService.signup(username, email, password);
  }

  @Post('signupvet')
  async signupvet(
    @Body('username') username: string,
    @Body('email') email: string,
    @Body('password') password: string
  ): Promise<User> {
    return this.userService.signupvet(username, email, password);
  }

  @Post('resend-otp')
  async resendOtp(@Body('email') email: string): Promise<{ message: string }> {
    if (!email) {
      throw new BadRequestException('Email is required');
    }

    return this.userService.resendOtp(email);
  }

  @Post('verify-otp')
  async verifyOtp(@Body('email') email: string, @Body('otp') otp: string): Promise<User> {
    return this.userService.verifyOtp(email, otp);
  }

  @Post('login')
  async login(@Body('email') email: string, @Body('password') password: string): Promise<User> {
    return this.userService.login(email, password);
  }

  @Put('/:id/verify')
  async verifyVet(@Param('id') id: string) {
    try {
      const updatedVet = await this.userService.verifyVetById(id);
      return {
        message: 'Vet verified successfully',
      };
    } catch (error) {
      // Handle error
    }
  }
}
```

```
        vet: updatedVet,
      };
    } catch (error) {
      return {
        message: 'Error verifying vet',
        error: error.message,
      };
    }
  }
}
```

---

LISTING 3: User Authentication in NestJS

## Booking an Appointment by User

---

```
import { Body, Controller, Post } from '@nestjs/common';
import { AppointmentService } from '../appointment.service';

@Controller('appointments')
export class AppointmentController {
  constructor(private readonly appointmentService: AppointmentService) {}

  @Post()
  async bookAppointment(@Body() body: {
    name: string;
    email: string;
    vetName: string;
    vetEmail: string;
    location: string;
    timeAndDate: string;
  }) {
    const appointment = await this.appointmentService.createAppointment(body);
    return {
      message: 'Appointment booked successfully!',
      appointment,
    };
  }
}
```

---

LISTING 4: Booking Appointment in NestJS

## Model Definition and Training

---

```

from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dropout, Dense, Input
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.callbacks import ModelCheckpoint

# Define input shape
input_shape = (256, 256, 3)

# Initialize the base model with the input shape
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=input_shape)
base_model.trainable = False # Freeze the base model

# Create input layer
inputs = Input(shape=input_shape)

# Pass inputs through the base model
x = base_model(inputs, training=False) # Set training=False to avoid updating batch norm layers

# Add custom layers on top
x = GlobalAveragePooling2D()(x)
x = Dropout(0.15)(x)
x = Dense(1024, activation='relu')(x)
outputs = Dense(2, activation='softmax')(x) # Output layer for binary classification

# Define the full model
model = Model(inputs, outputs)

# View model summary
model.summary()

# Setting variables for the model
batch_size = 32
epochs = 10

# Separating Training and Testing Data
train_generator = datagenerator["train"]
valid_generator = datagenerator["valid"]

# Calculating variables for the model
steps_per_epoch = train_generator.n // batch_size
validation_steps = valid_generator.n // batch_size

print("steps_per_epoch :", steps_per_epoch)
print("validation_steps :", validation_steps)

# Compile the model with appropriate loss function, optimizer, and metrics
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy', # Use 'binary_crossentropy' if you have 2 classes and it's
    metrics=['accuracy']
)

# File Path to store the trained models
filepath = "./model_{epoch:02d}-{val_accuracy:.2f}.keras"

# Using the ModelCheckpoint function to train and store all the best models

```

```
checkpoint1 = ModelCheckpoint(filepath, monitor='val_accuracy', verbose=1, save_best_only=True,

callbacks_list = [checkpoint1]

# Training the Model
history = model.fit(
    train_generator,
    epochs=epochs,
    steps_per_epoch=steps_per_epoch,
    validation_data=valid_generator,
    validation_steps=validation_steps,
    callbacks=callbacks_list
)
```

---

LISTING 5: Model Definition and Training with MobileNetV2

# GitHub Repository Link

The full source code for the VDerm-X system is available at:

<https://github.com/a-jay262/fyp2>